
RL-SPH: Learning to Achieve Feasible Solutions for Integer Linear Programs

Tae-Hoon Lee¹ Min-Soo Kim¹

Abstract

Primal heuristics play a crucial role in quickly finding feasible solutions for NP-hard integer linear programming (ILP). Although *end-to-end learning*-based primal heuristics (E2EPH) have recently been proposed, they are typically unable to independently generate feasible solutions. To address this challenge, we propose RL-SPH, a novel reinforcement learning-based start primal heuristic capable of independently generating feasible solutions, even for ILP involving non-binary integers. Empirically, RL-SPH rapidly obtains high-quality feasible solutions with a 100% feasibility rate, achieving on average a $28.6\times$ lower primal gap and a $2.6\times$ lower primal integral compared to existing start primal heuristics.

1. Introduction

Combinatorial optimization (CO) involves mathematical optimization that aims to minimize or maximize a specific objective function (Mazyavkina et al., 2021). When both the objective function and the constraints of CO are linear, the problem is referred to as linear programming (LP) (Bengio et al., 2021). Furthermore, if the variables in LP are required to take integer values, it becomes an integer linear programming (ILP) (Bertsimas & Tsitsiklis, 1997). ILP has been widely applied to real-world scenarios such as logistics (Kweon et al., 2024), the vehicle routing problem (Toth & Vigo, 2002), and path planning (Zuo et al., 2020).

Since ILP is NP-hard, heuristic approaches have attracted significant attention (Berthold, 2006). Primal heuristics aim to quickly find feasible solutions (Berthold, 2006; Shoja & Axehill, 2023), in contrast to methods that aim for optimality (Cantürk et al., 2024). Traditional primal heuristics often rely on expert knowledge, requiring significant manual effort (Bengio et al., 2021). Recently, ML-based primal

heuristics have been proposed (Nair et al., 2020; Shen et al., 2021; Yoon, 2022; Han et al., 2023; Cantürk et al., 2024; Huang et al., 2024; Zeng et al., 2024; Liu et al., 2025a; Geng et al., 2025), which fall under the category of *end-to-end learning* (Bengio et al., 2021; Han et al., 2023), as they learn common patterns across ILP instances and directly generate solutions. Figure 1(a) illustrates how existing *end-to-end learning*-based primal heuristics (E2EPH) participate in the process of finding feasible solutions. A trained ML model generates a *partial solution* over integer variables, which is then passed to an external ILP solver (e.g., Gurobi and SCIP) to obtain a feasible solution for the subproblem.

E2EPH methods integrated with ILP solvers efficiently find high-quality solutions by reducing the search space. Despite these advances, ensuring feasibility remains a major challenge, as inaccurate ML predictions can lead to constraint violations (Han et al., 2023), posing a significant obstacle to solving ILP. Recent studies (Han et al., 2023; Huang et al., 2024; Liu et al., 2025a) have sought to mitigate this risk by adopting trust regions rather than strictly fixing variables (Nair et al., 2020; Yoon, 2022). However, they rely on external solvers to attain feasibility. Several E2EPH have been proposed to eliminate this reliance (Zeng et al., 2024; Geng et al., 2025), but they still struggle to obtain feasible solutions independently. This limitation underscores the need for a new class of E2EPH that can independently produce feasible solutions, known as *start primal heuristics* (SPH), which attempt to convert infeasible solutions into feasible ones (Berthold, 2006) (see Appendix D.2).

The infeasibility caused by inaccurate ML predictions can be more pronounced for non-binary integer (hereafter, integer) variables due to their wider value range compared to binary variables. Many real-world problems involve integer variables (e.g., logistics (Kweon et al., 2024) and maritime transportation (Papageorgiou et al., 2014)). However, existing E2EPH studies have primarily focused on binary variables (Han et al., 2023; Huang et al., 2024; Zeng et al., 2024; Liu et al., 2025a; Geng et al., 2025), highlighting the need for E2EPH capable of effectively handling integer variables.

This work focuses on achieving feasibility in ILP, a critical prerequisite for optimality. For instance, popular meta-heuristics such as local branching (Liu et al., 2022) and large neighborhood search (LNS) (Huang et al., 2023; Liu et al.,

¹School of Computing, KAIST, Daejeon, Republic of Korea. Correspondence to: Tae-Hoon Lee <th.lee@kaist.ac.kr>, Min-Soo Kim <minsoo.k@kaist.ac.kr>.

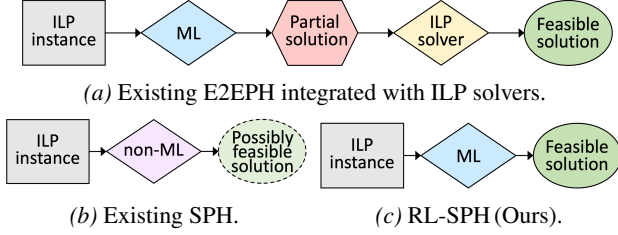


Figure 1. Comparison among *end-to-end learning*-based primal heuristics (E2EPH), start primal heuristics (SPH), and ours.

2025b) require an initial feasible solution to initiate their optimization process. Given that reaching optimality remains a significant challenge in the ML for CO field (Son et al., 2023), ensuring feasibility becomes an essential first step in solving ILP. However, feasibility is not typically guaranteed by established primal heuristics (Shoja & Axehill, 2023). While finding feasible solutions for ILP is not trivial due to its theoretically NP-hard nature, it poses an even greater challenge in the context of ML (Bengio et al., 2021).

To tackle this challenge—where ML must generate feasible solutions for ILP independently of external solvers—we propose a novel **Reinforcement Learning-based Start Primal Heuristic**, called **RL-SPH**. Figure 1(c) depicts how RL-SPH generates feasible solutions for ILP. We design an ILP-specific RL framework, in which the agent learns variable-constraint relationships to discover feasible solutions. Specifically, the agent learns to decide whether to change the value of each variable, guided by reward signals derived from the degree of constraint violations and the quality of the solution. We also introduce ILP-GT, a Graph Transformer specifically designed for ILP to effectively model long-range dependencies between variables (see Appendix D.4). Our main contributions are as follows, and Appendix E further contextualizes our research.

- We propose a novel RL-based start primal heuristic, RL-SPH, which learns to independently generate high-quality feasible solutions for ILP. To the best of our knowledge, RL-SPH is the first E2EPH to establish theoretical reward-feasibility alignment, a conditional feasibility guarantee.
- We design an RL framework tailored for ILP, wherein the ILP-GT agent interacts to acquire problem-solving capabilities in an end-to-end fashion. By leveraging our feasibility-aware search strategy to narrow the search space, the agent efficiently learns to achieve ILP-feasible solutions via strong reward signals.
- We demonstrate that RL-SPH achieves a 100% feasibility rate (FR) across five CO benchmarks—including non-binary ILP—while outperforming seven baselines comprising four established SPH and three E2EPH methods. Our evaluations include a wide array of tests: cross-problem generalization, robustness to distributional shifts, tests on MIPLIB instances, and solver integration, alongside hyperparameter and qualitative analyses.

2. Preliminaries

2.1. Integer Linear Programming

Integer linear programming (ILP) is an optimization problem that minimizes or maximizes a linear objective function, while satisfying linear constraints and integrality constraints on decision variables (Bertsimas & Tsitsiklis, 1997). A standard form of an ILP instance can be formulated as follows:

$$\text{minimize } \mathbf{c}^\top \mathbf{x} \quad (1a)$$

$$\text{subject to } \mathbf{A}\mathbf{x} \leq \mathbf{b} \quad (1b)$$

$$x_i \in \mathbb{Z}, \quad \forall i \quad (1c)$$

$$l_i \leq x_i \leq u_i, \quad \forall i \quad (1d)$$

where $\mathbf{x} \in \mathbb{R}^n$ is a column vector of n decision variables, $\mathbf{c} \in \mathbb{R}^n$ is a column vector of the objective coefficients, $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the constraint coefficient matrix, $\mathbf{b} \in \mathbb{R}^m$ is a column vector of the right-hand side of the constraints, and l_i/u_i denote the lower/upper bounds for each decision variable x_i . A solution $\mathbf{x}$ is said to be *feasible* if it satisfies all constraints (Eqs. 1b–1d). ILP aims to find a feasible solution that minimizes $obj = \mathbf{c}^\top \mathbf{x}$ (Eq. 1a) in the case of a minimization problem, which defines the optimal solution. Additional properties are provided in Appendix D.1.

2.2. Actor–Critic Reinforcement Learning

The Actor–Critic (AC) framework (Mnih et al., 2016) is a foundational RL paradigm that consists of two components: an actor, which selects actions, and a critic, which evaluates them. In this framework, the actor aims to maximize the expected reward by learning a policy $\pi_\theta(\mathcal{A} \mid \mathcal{S})$ that maps an observation $\mathcal{S}$ to a probability distribution over actions $\mathcal{A}$, while the critic evaluates $\mathcal{S}$ using a value function $V_\theta(\mathcal{S})$. Motivated by the effectiveness of AC for CO problems (Chen & Tian, 2019; Hubbs et al., 2020; Zong et al., 2022), we design an ILP-tailored AC to train our ILP-GT.

2.3. Neighborhood Search for CO

Local search (LS) heuristic is widely used for exploring the neighborhood of a current solution for CO problems (Hillier & Lieberman, 2015). Although LS may encounter local optima, empirical studies have shown it to be effective in quickly identifying high-quality solutions (Bertsimas & Tsitsiklis, 1997). Classic LS typically perturbs a single variable at a time, whereas exploring larger neighborhoods has been shown to improve effectiveness (Shaw, 1998). In contrast, with our newly designed search strategy, RL-SPH allows multiple variables to change simultaneously as shown in Figure 2, thereby exploring a broader neighborhood. Our search strategy resembles RENS in that it solves subproblems by fixing a set of variables and also LNS in exploring large neighborhoods. However, unlike RENS and RL-SPH, LNS requires an *initial feasible solution*, limiting its use as a start primal heuristic (Berthold, 2006).

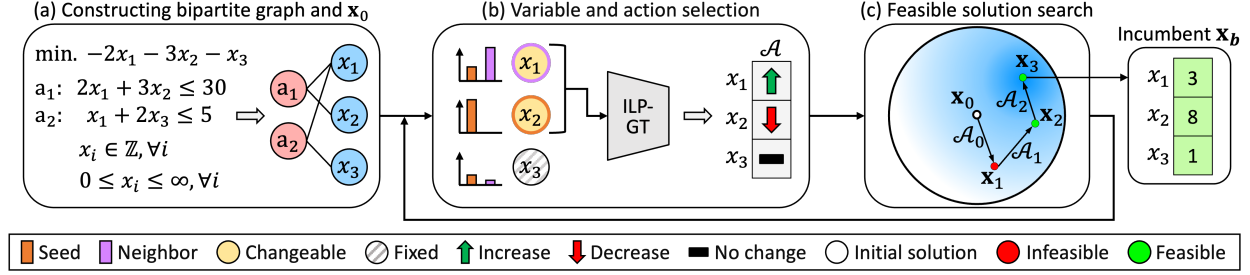


Figure 2. The overview of RL-SPH.

3. Methodology

Figure 2 illustrates the overall process of RL-SPH. Given an ILP instance, RL-SPH constructs a bipartite graph (see Appendix D.3) and an initial solution $\mathbf{x}_0$. At each timestep, it selects $\tilde{n} (= 2 \lceil \log_2 n \rceil)$ changeable variables as input to the trained agent. The agent selects actions expected to yield high rewards and generates a new solution. As the process repeats, the best feasible solution found so far (i.e., the incumbent) $\mathbf{x}_b$ is updated whenever $\mathbf{x}_{t+1}$ is both feasible and improves upon $\mathbf{x}_b$.

3.1. Reinforcement Learning for ILP

Our RL framework for ILP aims to train an agent to make decisions that maximize rewards while interacting with a given ILP instance M . Figure 3 depicts how the RL agent interacts with M , where $\mathcal{S}_t$, $\mathcal{A}_t$, and $\mathcal{R}_{t,\text{total}}$ denote the observation, the set of actions, and the total reward at timestep t , respectively. Using $\mathcal{A}_t$, the agent updates the solution $\mathbf{x}_{t+1}$ for n variables. This update affects the left-hand side of the constraints lhs_{t+1} , the feasibility state vector $\mathbf{f}_{t+1}$, and the objective value obj_{t+1} . At the next timestep, the agent receives a new observation $\mathcal{S}_{t+1}$ changed by $\mathcal{A}_t$ and selects a new action set $\mathcal{A}_{t+1}$ to maximize rewards. By comparing the estimated reward with the actual reward $\mathcal{R}_{t+1,\text{total}}$ obtained from $\mathcal{A}_{t+1}$, the agent refines its policy π .

3.1.1. ACTION

At timestep t , the agent selects a set of actions $\mathcal{A}_t = (a_{t,1}, \dots, a_{t,n})$ for n variables based on $\mathcal{S}_t$. For each variable, the agent can take one of three actions: increase, no change, or decrease, as depicted in Figure 3(a). The magnitude of change for both increases and decreases is set to 1, as discussed in Appendix I.7. Non-changeable variables are treated as fixed (i.e., no change), as shown in Figure 2(b).

3.1.2. OBSERVATION

We define the observation $\mathcal{S}_t = (\mathbf{x}_t, \mathbf{f}_t, \text{obj}_t)$. The solution $\mathbf{x}_t$ is obtained by updating variable values based on the previous actions $\mathcal{A}_{t-1}$. For example, if $\mathcal{A}_{t-1} = (1, -1, 0)$, then $\mathbf{x}_{t-1}$ in Figure 3(b) is updated to $\mathbf{x}_t = (x_{t,1}, x_{t,2}, x_{t,3}) = (4, 8, 0)$. Using $\mathbf{x}_t$, the new $\text{lhs}_t = \mathbf{A}\mathbf{x}_t$ and $\text{obj}_t = \mathbf{c}^\top \mathbf{x}_t$ are calculated. Each element of $\mathbf{f}_t (= \mathbf{b} - \text{lhs}_t)$ indicates whether the corresponding constraint is satisfied by $\mathbf{x}_t$. Non-

negative elements in $\mathbf{f}_t$ indicate satisfied constraints, while negative ones indicate violations. For example, in Figure 3, $\mathbf{x}_{t+1}$ yields $\text{lhs}_{t+1} = (34, 3)$ for constraints $\mathbf{a}_1$ and $\mathbf{a}_2$. Since $\mathbf{f}_{t+1} = \mathbf{b} - \text{lhs}_{t+1} = (30, 5) - (34, 3) = (-4, 2)$, $\mathbf{x}_{t+1}$ violates $\mathbf{a}_1$ but satisfies $\mathbf{a}_2$.

3.1.3. REWARD

We design reward functions to guide the agent in selecting actions that maximize the total reward $\mathcal{R}_{t,\text{total}}$, as follows:

$$\mathcal{R}_{t,\text{total}} = \mathcal{R}_{t,\text{opt}} + \mathcal{R}_{t,\text{explore}} \quad (2)$$

$$\mathcal{R}_{t,\text{opt}} = \begin{cases} \mathcal{R}_{t,p1}, & \text{if agent is in } phase1, \\ \mathcal{R}_{t,p2}, & \text{otherwise.} \end{cases} \quad (3)$$

To fulfill the goal of start primal heuristics—obtaining high-quality feasible solutions quickly—we decompose the process into two phases: the rapid discovery of an initial feasible solution in *phase1* and the search for higher-quality feasible solutions in *phase2*. *phase1* lasts until the first feasible solution is found, while *phase2* begins thereafter. The reward functions for each phase are defined as follows:

$$\mathcal{R}_{t,p1} = \begin{cases} \mathcal{R}_{t,\text{bound}}, & \text{if } \neg \mathcal{B}_t \wedge \mathcal{C}_t \wedge \mathcal{O}_t, \\ \mathcal{R}_{t,\text{bound}} - \Delta \text{obj}_t, & \text{if } \neg \mathcal{B}_t \wedge \mathcal{C}_t \wedge \neg \mathcal{O}_t, \\ \mathcal{R}_{t,F} + \Delta \text{obj}_t, & \text{if } \mathcal{B}_t \wedge \mathcal{C}_t \wedge \mathcal{O}_t, \\ \mathcal{R}_{t,F} - \Delta \text{obj}_t, & \text{if } \neg \mathcal{C}_t \wedge \neg \mathcal{O}_t, \\ \mathcal{R}_{t,F}, & \text{otherwise.} \end{cases} \quad (4)$$

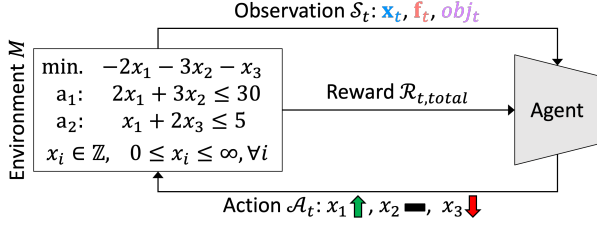
$$\mathcal{R}_{t,p2} = \begin{cases} \Delta \text{obj}_t, & \text{if } \mathbf{x}_{t+1} \in \mathcal{F} \wedge \text{obj}_{t+1} < \text{obj}_b, \\ -\Delta \text{obj}_t \cdot \alpha, & \text{if } \mathbf{x}_{t+1} \in \mathcal{F} \wedge \text{obj}_{t+1} \geq \text{obj}_b, \\ \mathcal{R}_{t,F}, & \text{if } \mathbf{x}_{t+1} \notin \mathcal{F} \wedge \text{obj}_{t+1} < \text{obj}_b, \\ \mathcal{R}_{t,F} \cdot \alpha & \text{otherwise.} \end{cases} \quad (5)$$

$$\mathcal{R}_{t,F} = \mathcal{R}_{t,\text{bound}} + \frac{1}{\sqrt{\tilde{n}}} \mathcal{R}_{t,\text{const}} \quad (6)$$

$$\mathcal{R}_{t,\text{bound}} = - \sum_{i=1}^n \mathbb{I}(x_{t+1,i} \notin [l_i, u_i]) \quad (7)$$

$$\mathcal{R}_{t,\text{const}} = \sum_{j=1}^m \min(f_{t+1,j}, 0) - \min(f_{t,j}, 0) \quad (8)$$

where $\mathcal{B}_t$ is the condition $x_{t+1,i} \in [l_i, u_i]$ for all i , $\mathcal{C}_t$ is $\mathcal{R}_{t,\text{const}} > 0$, $\mathcal{O}_t$ is $\text{obj}_{t+1} < \text{obj}_b$, $\mathcal{F}$ is the feasible region,



(a) Diagram of RL framework for ILP.

	x_1	x_2	x_3		f_1	f_2		obj_{t-1}
$\mathbf{x}_{t-1}$:	3	9	0		-3	2		-33
$\mathbf{x}_t$:	4	8	0		-2	1		-32
$\mathbf{x}_{t+1}$:	5	8	-1		-4	2		-33

(b) Examples of observation for Figure 3(a).

Figure 3. Reinforcement learning for ILP.

$\Delta obj_t = |obj_{t+1} - obj_t| / \max(|\mathbf{c}|)$, α is a toward-optimal bias, $\mathbb{I}$ is the indicator function, $x_{t,i}$ of $\mathbf{x}_t$ is the value of the i -th variable at t , $f_{t,j}$ is the element of $\mathbf{f}_t$ for the j -th linear constraint, and $\tilde{n}$ is the number of changeable variables.

Finding a feasible solution is a prerequisite for improving the incumbent. Thus, our primary goal is to find a feasible solution that satisfies all constraints. The feasibility reward $\mathcal{R}_{t,F}$, used in Eqs. 4 and 5, is computed based on the variable bounds and linear constraints. The bound reward $\mathcal{R}_{t,bound}$ imposes a penalty proportional to the number of variables that violate their bounds. For example, in Figure 3(b), $\mathcal{R}_{t,bound} = -1$ since $l_i = 0$ and $x_{t+1,3} = -1$. The constraint reward $\mathcal{R}_{t,const}$ reflects the improvement (or deterioration) for each infeasible linear constraint. For example, in Figure 3(b), $\mathcal{R}_{t,const}$ is $[\{-4 - (-2)\} + \{0 - (0)\}] = -2$.

In *phase1*, the primary goal is to find the first feasible solution from an infeasible initial solution. Since improvements in constraint violations or objective values are meaningless when variable bounds are violated, satisfying the variable bounds should be prioritized above all else. Thus, positive values of $\mathcal{R}_{t,const}$ are ignored to prioritize satisfying the bounds (Cases 1, 2 in Eq. 4). In all other cases, the agent receives $\mathcal{R}_{t,F}$ during *phase1* (Cases 3, 4, 5 in Eq. 4). Achieving a better (i.e., lower) objective value in *phase1* leads to a stronger starting point for *phase2*. Accordingly, the agent receives a reward proportional to the changes in the objective value (Cases 2, 3, 4 in Eq. 4). However, a positive Δobj_t is given only when feasibility improves (Case 3 in Eq. 4), preserving the priority. A well-trained agent selects actions that maximize the rewards and is thus theoretically directed toward a feasible solution, grounded in the reward-feasibility alignment established in Proposition 1:

Proposition 1. Suppose $\mathbf{x}_t \notin \mathcal{F}$, $\mathcal{R}_{t,const} > 0$, and $\mathcal{R}_{t,bound} = 0$ for all $t < T$. Then $\mathbf{x}_T \in \mathcal{F}$.

Proof. Appendix A provides the proof of Proposition 1. $\square$

In *phase2*, the goal is to improve the incumbent solution $\mathbf{x}_b$. If the agent finds $\mathbf{x}_{t+1} \in \mathcal{F}$, it receives a reward proportional to the changes in the objective value (Cases 1, 2 in Eq. 5). If $\mathbf{x}_{t+1} \notin \mathcal{F}$, it is penalized for violations (Cases 3, 4 in Eq. 5). A suitable α promotes the agent to explore promising regions where $obj_{t+1} < obj_b$. Empirically, search performance degrades in the absence of the toward-optimal bias (i.e., $\alpha = 1$); thus, we set α to 2. Appendix H.4.1 presents an evaluation of the impact of toward-optimal bias.

Exploration must take precedence even over satisfying variable bounds; otherwise, the agent might remain stationary to avoid potential penalties. To discourage such behavior, we assign a heavy exploration penalty $\mathcal{R}_{t,explore} = -100$ only if $\mathbf{x}_{t+1} = \mathbf{x}_t$. The maximum bound penalty occurs when all changeable variables violate their bounds, expressed as $-\tilde{n} = -2 \lceil \log_2 n \rceil$. Exceeding -100 would require a bound penalty corresponding to $n > 10^{15}$, which is practically infeasible. Thus, we impose -100 if the agent does not move. Appendix F further details our reward system.

3.2. Graph Transformer for ILP

Figure 4 shows the architecture of our Graph Transformer, ILP-GT. The actor generates a sequence of actions conditioned on the selected $\tilde{n}$ variables and their structural information ($\mathbf{c}^\top |\mathbf{A}|$). To stabilize training, we scale $(\mathbf{c}^\top |\mathbf{A}|)$ to $[-1, 1]$ using equilibration scaling (Tomlin, 1975), which normalizes each constraint by its largest absolute coefficient while preserving problem equivalence. The scaled structural information is projected to the hidden size. A binary feature *bound_lim* is set to 1 if a variable value reaches or exceeds its bound, otherwise 0. For example in Figure 3, the *bound_lim* for $x_{t,1}$, $x_{t,2}$, and $x_{t,3}$ would be 0, 0, and 1, respectively. Since the raw variable values may be unbounded, we embed them using Periodic Embedding (PE) (Gorishniy et al., 2022), which has proven effective for numerical features in ML tasks (Kohavi et al., 1996; Pace & Barry, 1997). PE is formulated as $PE(z) = \oplus(\sin(\tilde{z}), \cos(\tilde{z}))$, where $\tilde{z} = [2\pi w_1 z, \dots, 2\pi w_k z]$ with scalar z and trainable w_i .

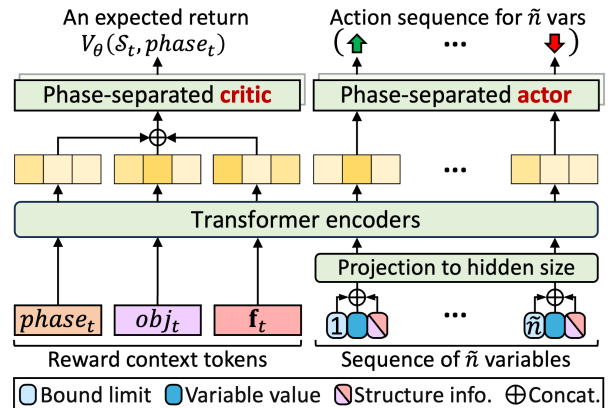


Figure 4. The architecture of ILP-GT.

Each final variable node is obtained by concatenating its structural information, bnd_lim , and variable value embedding, followed by a projection to the hidden size.

The critic approximates an expected return using reward context tokens, which consist of a phase indicator $phase_t$, a PE-encoded obj_t , and a scaled f_t . The reward context is essential because the total reward is determined by changes in solution quality and feasibility specific to each phase (see Eq. 2). We also introduce *phase-separated actor* and *critic layers* to align with the two-phase reward designs. In both phases, all other layers share parameters, and the input/output configuration is consistent. f_t is scaled by $\sqrt{|b| + |b - lhs_t|}$ and projected to the hidden size. The reward context tokens and variable nodes comprise a sequence of length $\tilde{n} + 3$, which is fed into the Transformer encoders.

3.3. Feasibility-Aware Search Strategy

To explore the solution space efficiently, RL-SPH selects $\tilde{n}(= p + q)$ changeable variables that are more likely to improve feasibility at each step t . The selection process comprises two steps, varying by phase. In *phase1*, RL-SPH stochastically selects p seed variables that frequently appear in violated linear constraints. Then, it selects the top q neighbor variables that appear most often with the seed variables in the same constraints. For example, in Figure 3, constraint a_1 is infeasible at step t ; thus, x_1 and x_2 are candidates for selection. If $p = q = 1$ and x_2 is selected as the seed, then x_1 is selected as a neighbor since it shares the same constraint with the seed. Consequently, x_1 and x_2 become changeable, while x_3 remains fixed. In *phase2*, RL-SPH stochastically selects p seed variables with a relatively low risk of abruptly depleting the available slack within the linear constraints. Neighbor selection in *phase2* follows the same procedure as in *phase1*. To limit the growth of input size in ILP-GT, we set p and q to $\lceil \log_2 n \rceil$. The pseudo-code appears in Algorithm 2, Appendix B.

Algorithm 1 outlines the overall procedure of our search strategy, which is used during both training and testing. RL-SPH first selects the variables to be changed with the feasibility-aware selection (Line 1), which are then used by the actor π_θ to predict $\mathcal{A}_t$ (Line 2). Based on $\mathcal{A}_t$, the agent obtains the next observation $\mathcal{S}_{t+1}$ (Lines 3-4). The agent receives a reward determined by the quality of $\mathbf{x}_{t+1}$ and the degree of feasibility improvement (Line 5). If $\mathbf{x}_{t+1}$ is a better feasible solution, both obj_b and $\mathbf{x}_b$ are updated (Lines 6-8). To guide exploration, we restrict movement to areas where further search is unnecessary (e.g., bound violations) (Lines 9-11). In *phase1*, the agent explores the solution space freely unless variable bounds are violated (Lines 9-10). In *phase2*, movements are rolled back unless a better feasible solution is discovered (Lines 9-10). The algorithm returns $\mathcal{R}_{t,\text{total}}$, $\mathcal{S}_{t+1}$, $\mathbf{x}_b$, and obj_b for the next step (Line 12). Appendix I.4 provides a discussion of our search strategy.

Algorithm 1 Feasibility-aware solution search of RL-SPH

Input: instance M , actor layer π_θ , $phase_t$, observation $\mathcal{S}_t = (\mathbf{x}_t, f_t, obj_t)$, incumbent solution $\mathbf{x}_b$, and incumbent value obj_b

Output: reward $\mathcal{R}_{t,\text{total}}$, new observation $\mathcal{S}_{t+1}$, $\mathbf{x}_b$, and obj_b

```

1:  $\tilde{\mathcal{S}}_t \leftarrow \text{select\_vars}(M, \mathcal{S}_t, phase_t)$       {See Alg. 2}
2:  $\mathcal{A}_t \leftarrow \pi_\theta(\tilde{\mathcal{S}}_t, phase_t)$ 
3:  $\mathbf{x}_{t+1} \leftarrow \text{move}(\mathbf{x}_t, \mathcal{A}_t)$ 
4:  $\mathcal{S}_{t+1} \leftarrow \text{observe}(M, \mathbf{x}_{t+1})$ 
5:  $\mathcal{R}_{t,\text{total}} \leftarrow \text{reward}(M, \mathcal{S}_{t+1}, \mathcal{S}_t, obj_b, phase_t)$ 
6: if  $\mathbf{x}_{t+1} \in \mathcal{F}$  and  $obj_{t+1} < obj_b$  then
7:    $obj_b \leftarrow obj_{t+1}$ 
8:    $\mathbf{x}_b \leftarrow \mathbf{x}_{t+1}$ 
9: else if ( $phase_t = 1$  and  $x_i \notin [l_i, u_i], \exists i$ )
   or  $phase_t = 2$  then
10:   $\mathcal{S}_{t+1} \leftarrow \mathcal{S}_t$ 
11: end if
12: return  $\mathcal{R}_{t,\text{total}}, \mathcal{S}_{t+1}, \mathbf{x}_b, obj_b$ 

```

3.4. Learning Algorithm

During training, our agent begins with an initial solution $\mathbf{x}_0$ obtained via either *LP-relaxation* or *random assignment*. If a more sophisticated initialization method is available, it can be readily integrated into RL-SPH. At timestep t , the agent observes $\tilde{\mathcal{S}}_t$ with $\tilde{n}$ variables. Afterwards, it selects $\mathcal{A}_t$ using $\pi_\theta(\mathcal{A}_t | \tilde{\mathcal{S}}_t, phase_t)$. Upon executing $\mathcal{A}_t$, the ILP environment returns the $\mathcal{R}_{t,\text{total}}$ and $\mathcal{S}_{t+1}$. The actor is trained to encourage actions that yield $\mathcal{R}_{t,\text{total}} > V_\theta(\mathcal{S}_t, phase_t)$ and to discourage those that yield lower rewards. The critic is trained to minimize the gap between $\mathcal{R}_t$ and $V_\theta(\mathcal{S}_t, phase_t)$ to provide accurate feedback to the actor. The agent stays in *phase1* for a predefined number of steps to ensure sufficient training, even after finding the first feasible solution. Once the step limit is reached, it moves to a new instance. Appendix C details the full training procedure.

4. Experiments

We validate the feasibility performance of RL-SPH through comprehensive experiments. First, we compare RL-SPH with widely-adopted start primal heuristics to demonstrate that it achieves the primary goal of quickly finding high-quality feasible solutions. Second, we assess RL-SPH against representative E2EPH methods, alongside the top-performing baselines from the first experiment. Third, we verify that our reward formulation effectively induces a directionality toward feasibility. Fourth, we evaluate the generalizability of RL-SPH in cross-problem settings. Fifth, we conduct ablation studies to assess the effectiveness of our feasibility-aware search strategy and ILP-GT components. Finally, Appendix H provides supplemental analyses, including tests on MIPLIB benchmarks, integration with ILP solvers, hyperparameter analyses, and qualitative analyses.

4.1. Experimental Setup

4.1.1. BENCHMARKS

We conduct experiments on five NP-hard ILP benchmarks commonly used in prior works (Gasse et al., 2019; Qi et al., 2021; Han et al., 2023; Huang et al., 2023). For minimum vertex cover (MVC), we generate instances based on the Barabási-Albert random graph models (Albert & Barabási, 2002), with 3,000 nodes, yielding 3,000 variables and 11,931 constraints on average. We generate instances for independent set (IS) using the same model as MVC with 1,500 nodes. For set covering (SC) (Balas & Ho, 1980), we generate instances with 3,000 variables and 2,000 constraints. For combinatorial auction (CA) (Leyton-Brown et al., 2000), instances are generated with 4,000 items and 2,000 bids, resulting in 4,000 variables and 2,677 constraints on average. We generate instances with 2,000 integer variables and 2,000 constraints, adapted from Qi et al. (2021), denoted as non-binary integers (NBI). For each dataset, we generated 1,000 training instances and 100 test instances. Additional details on the datasets are given in Appendix G.1.

4.1.2. BASELINES

In the first experiment, we evaluate RL-SPH against baselines from four major SPH classes: FP, RENS, diving, and rounding (see Appendix D.2). For comprehensive coverage, we group 15 diving and 6 rounding variants into the Diving Heuristic Family (DHF) and Rounding Heuristic Family (RHF) baselines, respectively (see Appendix G.3). We use the built-in implementations from the open-source ILP solver SCIP (v8.1.0), with presolving and branching disabled to isolate heuristic performance. Each baseline exclusively enables its corresponding heuristics with default settings. All randomization parameters in SCIP are set to 0 for reproducibility. Each baseline runs until its termination condition is met, with a 1,000-second time limit. RL-SPH, which has no predefined termination condition, stops when all baselines complete their search. If RL-SPH terminates without a feasible solution, it is recorded as a failure. Unless otherwise stated, RL-SPH uses LP initialization, and its termination criteria follow those of the first experiment.

In the second experiment, we compare RL-SPH with three representative E2EPH methods: PAS (Han et al., 2023), DDIM (Zeng et al., 2024), and DiffILO (Geng et al., 2025). While DDIM and DiffILO are our primary baselines due to their explicit use of feasibility-related metrics, PAS serves as a proxy for the other E2EPH methods (Huang et al., 2024; Liu et al., 2025a) that lack such metrics. The E2EPH baselines were trained and evaluated using their respective benchmark-specific configurations. In contrast, RL-SPH utilized a consistent hyperparameter configuration across all datasets, as detailed in Appendix G.2. RL-SPH took an average of 30 minutes to train, a $14.7\times$ speedup over the E2EPH baselines (see Appendix H.1).

4.1.3. METRICS

We use four evaluation metrics for the main experiment: the *primal gap* (PG) (Berthold, 2013; Huang et al., 2023; Cantürk et al., 2024), the *primal integral* (PI) (Berthold, 2013; Gasse et al., 2022), the *feasibility rate* (FR), and the *first feasible solution time* (FT). PG quantifies how close a method’s incumbent value obj_b is to the best-known solution (BKS), computed as $PG(obj_b) = \frac{|obj_b - BKS|}{\max(|obj_b|, |BKS|, \epsilon)}$. PI measures how quickly the incumbent improves toward BKS over time, defined as $PI(T) = \sum_{t=1}^T PG(obj_t)$, where $PG(obj_t) = 1$ if no feasible solution is available at actual wall-clock time t (in seconds). FR measures the ratio of instances in which a feasible solution is obtained. FT records the time taken to obtain the first feasible solution, measured in seconds. PG, PI, and FT are averaged only over instances where at least one feasible solution is found.

4.2. Comparison with Start Primal Heuristics

Table 1 summarizes the evaluation results across FR, PG, PI, and FT, with BKS identified among all methods. RL-SPH achieved 100% FR on all benchmarks, demonstrating its effectiveness in learning feasibility. Only RHF also achieved 100% FR on all benchmarks. We denote RL-SPH initialized with LP as RL-SPH(LP), and RL-SPH initialized with a randomly generated solution as RL-SPH(Random). Compared to the baselines with 100% FR, RL-SPH(LP) achieved a $28.6\times$ lower PG, a $2.6\times$ lower PI, and a $2.5\times$ faster FT on average. While RL-SPH(LP) exhibits a higher PG than RHF only on NBI, its comparable PI on NBI indicates that it finds high-quality feasible solutions at an earlier stage (see Figure 5). These results demonstrate RL-SPH(LP)’s superiority in rapidly discovering high-quality feasible solutions, confirming that it achieves the primary goal.

Notably, RL-SPH(Random) showed comparable performance to RL-SPH(LP), which begins from an LP-feasible solution. Furthermore, RL-SPH(Random) found the first feasible solution within 2 seconds for all datasets, attaining a $180\times$ average speedup in FT compared to 100% FR baselines. Random initialization enables an early start, accelerating the agent’s exploration for feasible solutions without waiting for LP solving. These results suggest that RL-SPH can find feasible solutions even from lower-quality initializations, motivating the development of initialization strategies that are more refined than random assignment but faster than LP-relaxation. RENS and DHF yielded similar results on IS and MVC, as both start from the same LP-feasible solution due to the same random seed for reproducibility. Moreover, the coefficient matrix $\mathbf{A}$ in these datasets is extremely sparse (density of 0.13% and 0.07%), resulting in LP-feasible solutions with few fractional values. In such cases, fixing variables has limited impact on the rest of the problem, leading both methods to produce similar solutions.

Table 1. Performance comparison against SPH baselines on five datasets. **Bold** and underline are used to indicate the best and second-best methods among those with 100% FR. Improvements are averaged over Dataset-Baseline pairs with 100% FR.

Dataset	Metric	DHF	FP	RHF	RENS	RL-SPH (Ours)	
						w/ Random	w/ LP
IS	FR (%) $\uparrow$	89	100	100	89	100	100
	PG (%) $\downarrow$	99.98 $\pm$ 0.05	20.60 $\pm$ 2.10	18.09 $\pm$ 2.74	99.98 $\pm$ 0.05	4.10 $\pm$ 2.25	0.14 $\pm$ 0.59
	PI $\downarrow$	19.1 $\pm$ 2.3	4.8 $\pm$ 0.7	4.9 $\pm$ 0.7	19.1 $\pm$ 2.3	3.8 $\pm$ 0.4	2.5 $\pm$ 0.2
	FT (sec) $\downarrow$	11.94 $\pm$ 2.27	0.22 $\pm$ 0.04	0.10 $\pm$ 0.00	11.94 $\pm$ 2.25	0.27 $\pm$ 0.07	0.55 $\pm$ 0.07
CA	FR (%) $\uparrow$	100	100	100	100	100	100
	PG (%) $\downarrow$	90.61 $\pm$ 1.68	18.09 $\pm$ 9.64	12.02 $\pm$ 9.97	81.70 $\pm$ 30.19	19.04 $\pm$ 8.20	3.82 $\pm$ 9.77
	PI $\downarrow$	105.7 $\pm$ 27.9	31.5 $\pm$ 15.6	23.0 $\pm$ 15.5	112.9 $\pm$ 27.7	36.7 $\pm$ 14.9	21.9 $\pm$ 13.8
	FT (sec) $\downarrow$	10.06 $\pm$ 0.90	10.66 $\pm$ 0.70	8.03 $\pm$ 0.51	102.17 $\pm$ 28.38	0.41 $\pm$ 0.08	8.26 $\pm$ 0.72
SC	FR (%) $\uparrow$	1	99	100	3	100	100
	PG (%) $\downarrow$	0.00 $\pm$ 0.00	0.34 $\pm$ 3.11	29.98 $\pm$ 12.38	0.00 $\pm$ 0.00	12.78 $\pm$ 16.25	9.67 $\pm$ 15.41
	PI $\downarrow$	60.0 $\pm$ 0.0	347.5 $\pm$ 73.5	321.5 $\pm$ 111.0	339.3 $\pm$ 212.8	101.3 $\pm$ 136.7	168.0 $\pm$ 124.9
	FT (sec) $\downarrow$	59.50 $\pm$ 0.00	344.75 $\pm$ 68.57	104.09 $\pm$ 16.97	338.50 $\pm$ 212.62	0.19 $\pm$ 0.07	<u>95.43</u> $\pm$ 7.39
MVC	FR (%) $\uparrow$	36	100	100	36	100	100
	PG (%) $\downarrow$	33.67 $\pm$ 0.70	6.76 $\pm$ 1.15	8.01 $\pm$ 1.20	33.67 $\pm$ 0.70	0.23 $\pm$ 0.45	0.81 $\pm$ 0.86
	PI $\downarrow$	46.0 $\pm$ 5.3	5.2 $\pm$ 1.2	6.1 $\pm$ 1.2	42.8 $\pm$ 5.2	4.2 $\pm$ 0.5	<u>5.0</u> $\pm$ 0.6
	FT (sec) $\downarrow$	40.37 $\pm$ 7.05	1.17 $\pm$ 0.24	0.20 $\pm$ 0.04	35.76 $\pm$ 6.95	1.63 $\pm$ 0.07	<u>1.55</u> $\pm$ 0.08
NBI	FR (%) $\uparrow$	100	100	100	100	100	100
	PG (%) $\downarrow$	0.77 $\pm$ 0.64	1.89 $\pm$ 0.08	0.00 $\pm$ 0.00	1.89 $\pm$ 0.08	28.32 $\pm$ 2.84	0.22 $\pm$ 0.06
	PI $\downarrow$	14.3 $\pm$ 1.3	20.8 $\pm$ 2.1	11.3 $\pm$ 1.1	108.5 $\pm$ 25.7	57.5 $\pm$ 16.7	11.3 $\pm$ 0.9
	FT (sec) $\downarrow$	12.15 $\pm$ 0.92	17.16 $\pm$ 1.60	10.71 $\pm$ 0.99	106.35 $\pm$ 25.44	0.10 $\pm$ 0.07	<u>10.32</u> $\pm$ 0.71

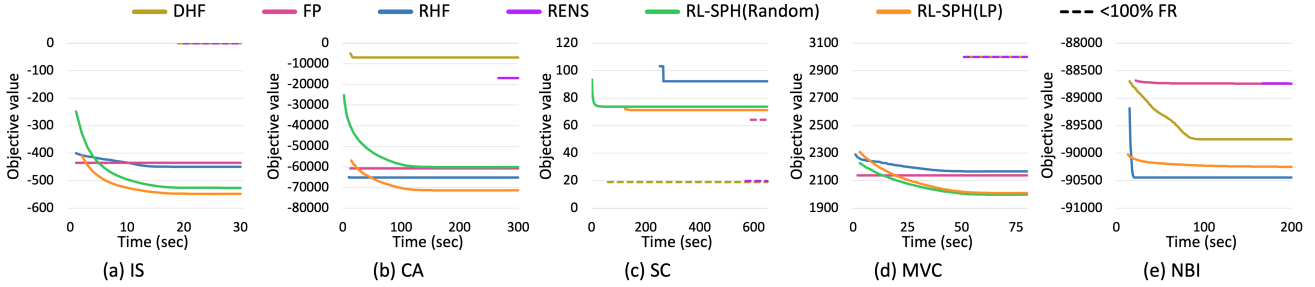


Figure 5. Illustration of the objective values of the compared methods over time (in seconds). Lines indicate valid averaged incumbent objective values, which are computed only when an incumbent objective value exists for all instances at a given time point. Dashed lines indicate that the FR is not 100%, and the averaging includes only instances for which at least one feasible solution was found.

4.3. Comparison with SOTA E2EPH

As shown in Table 2, only RL-SPH and RHF consistently achieved a 100% FR across all datasets, outperforming the other E2EPH baselines. PAS failed to find a feasible solution on CA and IS, aligning with the findings reported in the DiffiLO paper. DDIM exhibited higher FT than both RHF and all RL-SPH variants across all datasets. This latency stems not only from the LP-solving requirement for input feature construction but also from its slow inference speed. Specifically, DDIM conducted an average of 165, 149, and 76 trials for solution search on SC, CA, and IS within the time limit, respectively, yielding 0.5 to 2.5 trials per second. Moreover, DDIM attained higher PG than RHF on SC and CA, indicating inferior solution quality. Coupled with slow inference, this quality gap led to a $1.8\times$ higher PI for DDIM than for RHF. Thus, DDIM shows limitations in fulfilling the goal of quickly generating high-quality feasible solutions.

DiffiLO exhibited 0% FR on SC and CA despite making an average of 919,470 and 574,240 trials of a binomial distribution, respectively. We attribute the FR performance

gap between RL-SPH and DiffiLO to the prioritization between the objective function and the constraints. DiffiLO is trained using a soft-constrained loss function, where a penalty coefficient determines the relative influence of the objective and constraints, meaning that no explicit priority is enforced. Consequently, DiffiLO’s 0% FR on SC and CA can be attributed to the objective function dominating the constraints within its loss formulation. In contrast, RL-SPH enforces a clear priority between them: a positive reward for the objective value (i.e., $+\Delta obj_t$) is granted only when feasibility improves (Case 3 in Eq. 4).

Interestingly, RL-SPH was able to convert DiffiLO’s initially infeasible solutions into feasible ones on SC and CA. As shown in Figure 6, the curve for RL-SPH(DiffiLO) lies between those of RL-SPH(Random) and RL-SPH(LP) on SC and CA. This indicates that the initial solutions provided by DiffiLO offer a better starting point than random initialization, but a weaker one compared to LP-relaxation. Given that learning-based models provide rapid solution approximations, a refined inference approach can outperform LP initialization, as demonstrated by RL-SPH(DiffiLO) on IS.

Table 2. Performance comparison against representative E2EPH methods on SC, CA, and IS, which are the intersection of the benchmarks used in DDIM, DiffiLO, and our study. Since PAS does not provide hyperparameters for the SC benchmark, we exclude it from this experiment. *fail* indicates that the metrics cannot be computed because no feasible solution was obtained (i.e., FR = 0%). The time limit is set to 300, 300, and 30 seconds for SC, CA, and IS, respectively.

Dataset	Metric	Classical SPH baselines		E2EPH baselines			RL-SPH (Ours)		
		FP	RHF	PAS	DDIM	DiffiLO	w/ Random	w/ LP	w/ DiffiLO
SC	FR (%) $\uparrow$	5	100		99	0	100	100	100
	PG (%) $\downarrow$	0.00 $\pm$ 0.00	23.43 $\pm$ 4.02	—	59.30 $\pm$ 4.15	<i>fail</i>	4.65 $\pm$ 7.53	1.17 $\pm$ 1.90	3.47 $\pm$ 7.61
	PI $\downarrow$	243.6 $\pm$ 93.8	160.2 $\pm$ 17.8		232.3 $\pm$ 7.2	<i>fail</i>	16.0 $\pm$ 22.4	100.0 $\pm$ 8.2	13.5 $\pm$ 22.6
CA	FT (sec) $\downarrow$	242.62 $\pm$ 93.77	104.09 $\pm$ 16.97		133.00 $\pm$ 6.52		0.19 $\pm$ 0.07	95.43 $\pm$ 7.39	1.15 $\pm$ 0.25
	FR (%) $\uparrow$	100	100	0	100	0	100	100	100
	PG (%) $\downarrow$	22.81 $\pm$ 4.66	17.17 $\pm$ 2.74	<i>fail</i>	22.15 $\pm$ 2.65	<i>fail</i>	13.05 $\pm$ 2.07	0.33 $\pm$ 2.33	7.78 $\pm$ 2.15
IS	PI $\downarrow$	77.1 $\pm$ 13.5	58.6 $\pm$ 8.0	<i>fail</i>	122.5 $\pm$ 8.5	<i>fail</i>	71.5 $\pm$ 5.5	33.3 $\pm$ 4.7	52.9 $\pm$ 5.8
	FT (sec) $\downarrow$	10.66 $\pm$ 0.70	8.03 $\pm$ 0.51		71.42 $\pm$ 6.61		0.41 $\pm$ 0.08	8.26 $\pm$ 0.72	4.85 $\pm$ 0.27
	FR (%) $\uparrow$	100	100	0	100	100	100	100	100
IS	PG (%) $\downarrow$	34.32 $\pm$ 1.29	32.24 $\pm$ 1.94	<i>fail</i>	6.34 $\pm$ 1.07	0.35 $\pm$ 0.50	17.87 $\pm$ 1.86	15.35 $\pm$ 1.53	0.00 $\pm$ 0.00
	PI $\downarrow$	11.0 $\pm$ 0.4	10.8 $\pm$ 0.5	<i>fail</i>	21.2 $\pm$ 1.6	1.1 $\pm$ 0.2	8.5 $\pm$ 0.5	7.1 $\pm$ 0.4	1.0 $\pm$ 0.2
	FT (sec) $\downarrow$	0.22 $\pm$ 0.04	0.10 $\pm$ 0.00		20.11 $\pm$ 1.74	0.42 $\pm$ 0.16	0.27 $\pm$ 0.07	0.55 $\pm$ 0.07	0.66 $\pm$ 0.23

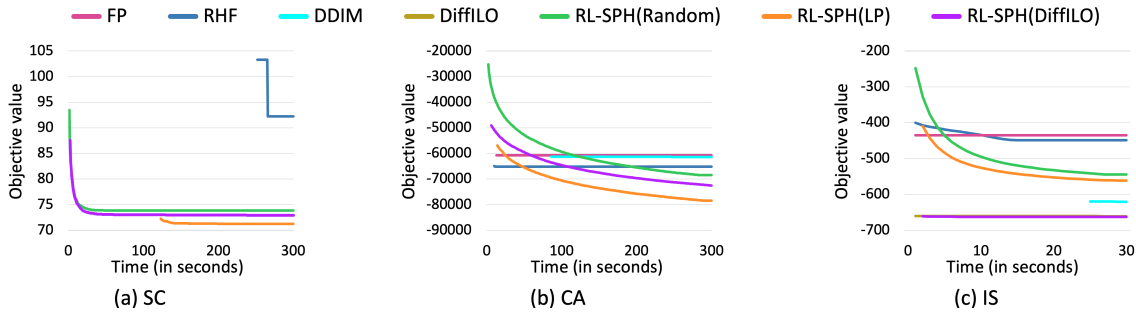


Figure 6. Illustration of the objective values of the compared methods over time (in seconds). Only cases with FR = 100% are displayed.

Meanwhile, the E2EPH baselines do not support integer variables, making them inapplicable to NBI. This limitation highlights the need for further research on solution approximation methods tailored for integer variables. Notably, RL-SPH(DiffiLO) was trained with LP initialization rather than with solutions generated by DiffiLO, demonstrating its adaptability to an alternative initialization method. Further comparative analysis is provided in Appendix H.1.

4.4. Analysis of Directionality Toward Feasibility

To bridge the gap between the theoretical analysis in Proposition 1 and practical performance, we provide empirical evidence in Table 3. The trained RL-SPH policy adheres to the prescribed condition (i.e., $\mathcal{R}_{t,\text{const}} > 0$ and $\mathcal{R}_{t,\text{bound}} = 0$ during *phase1*) in 95.2% of the trajectories, averaged across test datasets. The trained policy maintains steady progression toward feasibility, recovering from temporary stagnations in the remaining 4.8%. This strong directionality helps explain why RL-SPH consistently achieves 100% FR across five benchmarks. These results emphasize that our reward formulation successfully induces the reward-maximizing behavior that guides toward feasibility. Appendix H.5.1 provides visualizations that illustrate this behavior.

Table 3. Empirical adherence rates across five benchmarks.

Metric	IS	CA	SC	MVC	NBI
Adherence rate (%) $\uparrow$	98.7	91.5	98.4	99.1	88.4

4.5. Evaluation of Cross-Problem Generalization

Table 4 presents the performance of RL-SPH in cross-problem settings using IS and MVC. In this experiment, we matched the number of variables in MVC to that of IS ($n = 1,500$) to control for factors other than the problem type. RL-SPH trained solely on IS failed to produce any feasible solution on MVC test instances, and vice versa. However, a joint RL-SPH model trained on both problem types achieved a 100% FR on each test set. Moreover, the joint model outperformed RHF—the strongest baseline in terms of FR (see Tables 1 and 2)—in finding better incumbent values. Notably, the joint model also obtained feasible solutions for 17 diverse instances from the MIPLIB benchmark (Gleixner et al., 2021), which differ markedly from the training instances in terms of size (up to $67\times$ more variables), density (ranging from 0.1% to 100%), and variable types including integers (see Appendix H.2.1).

Table 4. Cross-problem performance of RL-SPH.

Test set	Metric	RHF	RL-SPH trained on		
			IS	MVC	IS+MVC
IS	FR (%) $\uparrow$	100	100	0	100
	PG (%) $\downarrow$	17.65 $\pm$ 2.87	0.00 $\pm$ 0.00	<i>fail</i>	15.55 $\pm$ 2.82
	PI $\downarrow$	4.6 $\pm$ 0.7	2.4 $\pm$ 0.2	<i>fail</i>	5.1 $\pm$ 0.6
MVC	FR (%) $\uparrow$	100	0	100	100
	PG (%) $\downarrow$	9.06 $\pm$ 1.58	<i>fail</i>	0.00 $\pm$ 0.00	7.13 $\pm$ 1.56
	PI $\downarrow$	2.8 $\pm$ 0.4	<i>fail</i>	1.8 $\pm$ 0.2	2.8 $\pm$ 0.4

Table 5. Ablation study on our solution search strategy. Uniform denotes variable selection uniformly at random, and F-aware refers to the feasibility-aware variable selection. PG_f is a variant of PG in which the incumbent value obj_b is replaced by the first feasible value, measuring the quality of the first feasible solution. $\mathcal{T}$ represents the number of search steps to find the first feasible solution.

Metric	IS		CA		SC		MVC		NBI	
	Uniform	F-aware	Uniform	F-aware	Uniform	F-aware	Uniform	F-aware	Uniform	F-aware
FR (%) $\uparrow$	100	100	0	100	100	100	100	100	100	100
PG_f (%) $\downarrow$	95.3 $\pm$ 1.6	41.4 $\pm$ 1.9	<i>fail</i>	31.4 $\pm$ 4.4	52.3 $\pm$ 4.8	39.0 $\pm$ 4.3	33.8 $\pm$ 0.7	16.0 $\pm$ 0.9	0.6 $\pm$ 0.2	0.3 $\pm$ 0.1
$\mathcal{T}$ $\downarrow$	399 $\pm$ 98	36 $\pm$ 2		59 $\pm$ 7	6 $\pm$ 1	4 $\pm$ 1	613 $\pm$ 109	68 $\pm$ 2	27 $\pm$ 6	46 $\pm$ 11

4.6. Ablation Study

Table 5 presents the effectiveness of our variable selection strategy across five datasets. Across all datasets, RL-SPH consistently obtained a feasible solution with the feasibility-aware selection strategy. In contrast, with the uniform selection, it often struggled to find a feasible solution quickly and even failed within the 100-second time limit on CA. For NBI, while RL-SPH required more steps with our search strategy, it achieved a higher-quality first feasible solution. These results demonstrate that our search strategy is effective in consistently finding high-quality feasible solutions.

Table 6 presents an ablation study on the ILP-GT architecture, where the time limit is set to 50 seconds. All models were trained under the same configuration. $\emptyset$ showed the lowest performance on both benchmarks and failed to find any feasible solution on CA. With TE, the model achieved a 100% FR on both benchmarks, indicating its effectiveness in learning feasibility. Combining all components achieved the best results in PG and PI, highlighting the importance of each component. Appendix H.5.2 provides a qualitative analysis of the long-range variable dependencies in TE.

Table 6. Ablation study on the ILP-GT architecture: RC (reward context), PSL (phase-separated layers), and TE (transformer encoder). $\emptyset$ denotes models without all components.

Dataset	Model	FR (%) $\uparrow$	PG (%) $\downarrow$	PI $\downarrow$
IS	$\emptyset$	100	41.23 $\pm$ 1.95	21.8 $\pm$ 0.9
	{RC, PSL}	100	33.98 $\pm$ 2.45	28.9 $\pm$ 2.4
	{TE, PSL}	100	1.10 $\pm$ 1.40	4.4 $\pm$ 0.7
	{TE, RC}	100	12.93 $\pm$ 1.96	11.7 $\pm$ 0.9
	{TE, RC, PSL}	100	0.62 $\pm$ 1.08	4.1 $\pm$ 0.5
CA	$\emptyset$	0	<i>fail</i>	<i>fail</i>
	{RC, PSL}	0	<i>fail</i>	<i>fail</i>
	{TE, PSL}	100	3.00 $\pm$ 2.27	12.4 $\pm$ 1.1
	{TE, RC}	100	1.63 $\pm$ 1.96	12.5 $\pm$ 1.0
	{TE, RC, PSL}	100	1.20 $\pm$ 1.53	11.8 $\pm$ 0.8

5. Discussion

As no E2EPH consistently achieves a 100% FR, we prioritize FR as the primary metric. In this context, RHF serves as a stronger baseline than the E2EPH baselines (see Appendix I.1). Our empirical evaluations confirm that RL-SPH significantly outperforms both widely-adopted start primal heuristics and SOTA E2EPH methods in generating high-quality feasible solutions (see Tables 1 and 2). Our literature review also provides evidence that existing E2EPH methods

struggle to achieve feasibility (see Appendix E.1), thereby highlighting the clear advantages of RL-SPH. Moreover, unlike most E2EPH methods, RL-SPH is designed to handle integer domains and attains a 100% FR on NBI. Notably, RL-SPH generalizes to unseen MIPLIB instances containing integer variables, despite being trained on binary-only instances (see Appendix H.2.1).

Beyond strong performance, RL-SPH offers significant advantages in training efficiency. Most E2EPH methods rely on supervised learning frameworks, necessitating near-optimal solutions as training labels that limit their practicality for NP-hard problems (Cantürk et al., 2024; Geng et al., 2025). In contrast, RL-SPH does not require such labels. Remarkably, it also achieves nearly 34 $\times$ faster training time than the most recent unsupervised learning-based E2EPH (see Appendix I.2). This efficiency stems from our search strategy, which enables focusing on informative regions where reward signals are dense (see Appendix I.3).

Furthermore, RL-SPH delivers robust results across five benchmarks with a consistent hyperparameter configuration, whereas prior E2EPH methods often require benchmark-specific tuning (see Appendix H.1). This enhances its practicality, facilitating easier training and deployment across diverse ILP settings with minimal adjustments. Appendix E.3 situates this general-purpose framework within a broader research context. RL-SPH provides a foundation for ML-based standalone solvers pursuing optimality, with further refinements (e.g., changes in action magnitude) outlined as future directions in Appendices I.4–I.8.

6. Conclusion

We proposed RL-SPH, which independently produces high-quality feasible solutions for ILP, grounded in a theoretically established reward-feasibility alignment. Empirically, RL-SPH attains a consistent 100% feasibility rate across five benchmarks, even for instances involving integer variables. It outperforms four SPH baselines—including RHF, which exhibits stronger feasibility performance than SOTA E2EPH methods—by achieving an average 28.6 $\times$ lower primal gap and 2.6 $\times$ lower primal integral. By addressing the feasibility bottleneck, our study resolves a critical gap left open by prior learning-based heuristics for ILP. This advance in feasibility represents a meaningful step toward addressing the major challenge of optimality in ML for CO.

Acknowledgements

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT)(No. RS-2024-00347471) and Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No. 2019-0-01267, GPU-based Ultrafast Multi-type Graph Database Engine SW), and the IITP(Institute of Information & Communications Technology Planning & Evaluation)-ITRC(Information Technology Research Center) grant funded by the Korea government(Ministry of Science and ICT)(IITP-2025-RS-2020-II201795).

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Albert, R. and Barabási, A.-L. Statistical mechanics of complex networks. *Reviews of modern physics*, 74(1):47, 2002.
- Alvarez, A. M., Louveaux, Q., and Wehenkel, L. A machine learning-based approximation of strong branching. *INFORMS Journal on Computing*, 29(1):185–195, 2017.
- Balas, E. and Ho, A. Set covering algorithms using cutting planes, heuristics, and subgradient optimization: a computational study. *Combinatorial optimization*, pp. 37–60, 1980.
- Bengio, Y., Lodi, A., and Prouvost, A. Machine learning for combinatorial optimization: a methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- Berthold, T. *Primal heuristics for mixed integer programs*. PhD thesis, Zuse Institute Berlin (ZIB), 2006.
- Berthold, T. Measuring the impact of primal heuristics. *Operations research letters*, 41(6):611–614, 2013.
- Berthold, T. and Hendel, G. Learning to scale mixed-integer programs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 3661–3668, 2021.
- Bertsimas, D. and Tsitsiklis, J. N. *Introduction to linear optimization*, volume 6. Athena Scientific Belmont, MA, 1997.
- Bestuzheva, K., Besançon, M., Chen, W.-K., Chmiela, A., Donkiewicz, T., Van Doornmalen, J., Eifler, L., Gaul, O., Gamrath, G., Gleixner, A., et al. The scip optimization suite 8.0. *arXiv preprint arXiv:2112.08872*, 2021.
- Cantürk, F., Varol, T., Aydoğan, R., and Özener, O. Ö. Scalable primal heuristics using graph neural networks for combinatorial optimization. *Journal of Artificial Intelligence Research*, 80:327–376, 2024.
- Chen, W., Tanneau, M., and Van Hentenryck, P. End-to-end feasible optimization proxies for large-scale economic dispatch. *IEEE Transactions on Power Systems*, 39(2): 4723–4734, 2023.
- Chen, X. and Tian, Y. Learning to perform local rewriting for combinatorial optimization. *Advances in neural information processing systems*, 32, 2019.
- Fischetti, M. and Lodi, A. Local branching. *Mathematical programming*, 98:23–47, 2003.
- Floudas, C. A. *Nonlinear and mixed-integer optimization: fundamentals and applications*. Oxford University Press, 1995.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. Exact combinatorial optimization with graph convolutional neural networks. *Advances in neural information processing systems*, 32, 2019.
- Gasse, M., Bowly, S., Cappart, Q., Charfreitag, J., Charlin, L., Chételat, D., Chmiela, A., Dumouchelle, J., Gleixner, A., Kazachkov, A. M., et al. The machine learning for combinatorial optimization competition (ml4co): Results and insights. In *NeurIPS 2021 competitions and demonstrations track*, pp. 220–231. PMLR, 2022.
- Geng, Z., Wang, J., Li, X., Zhu, F., Hao, J., Li, B., and Wu, F. Differentiable integer linear programming. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70, ICML’17*, pp. 1263–1272. JMLR.org, 2017.
- Gleixner, A., Hendel, G., Gamrath, G., Achterberg, T., Bastubbe, M., Berthold, T., Christophel, P. M., Jarck, K., Koch, T., Linderoth, J., Lübbecke, M., Mittelman, H. D., Ozyurt, D., Ralphs, T. K., Salvagnin, D., and Shinano, Y. Miplib 2017: Data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation*, 2021. doi: 10.1007/s12532-020-00194-3. URL <https://doi.org/10.1007/s12532-020-00194-3>.

- Gorishniy, Y., Rubachev, I., and Babenko, A. On embeddings for numerical features in tabular deep learning. *Advances in Neural Information Processing Systems*, 35: 24991–25004, 2022.
- Guieu, O. and Chinneck, J. W. Analyzing infeasible mixed-integer and integer linear programs. *INFORMS Journal on Computing*, 11(1):63–77, 1999.
- Gupta, P., Gasse, M., Khalil, E., Mudigonda, P., Lodi, A., and Bengio, Y. Hybrid models for learning to branch. *Advances in neural information processing systems*, 33: 18087–18097, 2020.
- Gurobi Optimization, L. Gurobi optimizer reference manual. <https://docs.gurobi.com/projects/optimizer>, 2025.
- Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., and Luo, X. A gnn-guided predict-and-search framework for mixed-integer linear programming. In *The Eleventh International Conference on Learning Representations*, 2023.
- He, H., Daumé, H., and Eisner, J. Learning to search in branch and bound algorithms. *Advances in neural information processing systems*, 27, 2014.
- Hillier, F. S. and Lieberman, G. J. *Introduction to operations research*. McGraw-Hill, 2015.
- Huang, T., Ferber, A. M., Tian, Y., Dilkina, B., and Steiner, B. Searching large neighborhoods for integer linear programs with contrastive learning. In *International conference on machine learning*, pp. 13869–13890. PMLR, 2023.
- Huang, T., Ferber, A. M., Zharmagambetov, A., Tian, Y., and Dilkina, B. Contrastive predict-and-search for mixed integer linear programs. In *Forty-first International Conference on Machine Learning*, 2024.
- Hubbs, C. D., Li, C., Sahinidis, N. V., Grossmann, I. E., and Wassick, J. M. A deep reinforcement learning approach for chemical production scheduling. *Computers & Chemical Engineering*, 141:106982, 2020.
- Hussain, K., Mohd Salleh, M. N., Cheng, S., and Shi, Y. Metaheuristic research: a comprehensive survey. *Artificial intelligence review*, 52(4):2191–2233, 2019.
- Hutter, F., Hoos, H. H., and Leyton-Brown, K. Sequential model-based optimization for general algorithm configuration. In *Proceedings of the 5th international conference on Learning and Intelligent Optimization*, pp. 507–523, 2011.
- Karmarkar, N. A new polynomial-time algorithm for linear programming. In *Proceedings of the sixteenth annual ACM symposium on Theory of computing*, pp. 302–311, 1984.
- Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pp. 5156–5165. PMLR, 2020.
- Khalil, E., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. Learning to branch in mixed integer programming. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.
- Kohavi, R. et al. Scaling up the accuracy of naive-bayes classifiers: A decision-tree hybrid. In *Kdd*, volume 96, pp. 202–207, 1996.
- Kostrikov, I. Pytorch implementations of reinforcement learning algorithms. <https://github.com/ikostrikov/pytorch-a2c-ppo-acktr-gail>, 2018.
- Kruber, M., Lübbecke, M. E., and Parmentier, A. Learning when to use a decomposition. In *International conference on AI and OR techniques in constraint programming for combinatorial optimization problems*, pp. 202–210. Springer, 2017.
- Kweon, O., Kim, B.-I., Lee, G., Im, H., Chung, C. Y., and Lim, O. K. Parcel delivery network optimization problem considering multiple hubs and consolidation of small-sized parcels. *Computers & Industrial Engineering*, 191: 110113, 2024.
- Labassi, A. G., Chételat, D., and Lodi, A. Learning to compare nodes in branch and bound with graph neural networks. *Advances in neural information processing systems*, 35:32000–32010, 2022.
- Leyton-Brown, K., Pearson, M., and Shoham, Y. Towards a universal test suite for combinatorial auction algorithms. In *Proceedings of the 2nd ACM conference on Electronic commerce*, pp. 66–76, 2000.
- Li, Q., Han, Z., and Wu, X.-M. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32(1), 2018.
- Lin, T., Wang, Y., Liu, X., and Qiu, X. A survey of transformers. *AI open*, 3:111–132, 2022.
- Liu, D., Fischetti, M., and Lodi, A. Learning to search in local branching. In *Proceedings of the aaai conference on artificial intelligence*, volume 36, pp. 3796–3803, 2022.

- Liu, F., Lin, X., Wang, Z., Zhang, Q., Xialiang, T., and Yuan, M. Multi-task learning for routing problem with cross-problem zero-shot generalization. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1898–1908, 2024.
- Liu, H., Wang, J., Geng, Z., Li, X., Zong, Y., Zhu, F., HAO, J., and Wu, F. Apollo-MILP: An alternating prediction-correction neural solving framework for mixed-integer linear programming. In *The Thirteenth International Conference on Learning Representations*, 2025a.
- Liu, W., Wang, A., Yang, W., and Shi, Q. Mixed-integer linear optimization via learning-based two-layer large neighborhood search. In *International Conference on Learning and Intelligent Optimization*, pp. 215–236. Springer, 2025b.
- Lourenço, H. R., Martin, O. C., and Stützle, T. Iterated local search. In *Handbook of Metaheuristics*, pp. 320–353. Springer US, Boston, MA, 2003.
- Mazyavkina, N., Sviridov, S., Ivanov, S., and Burnaev, E. Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research*, 134: 105400, 2021.
- Min, E., Chen, R., Bian, Y., Xu, T., Zhao, K., Huang, W., Zhao, P., Huang, J., Ananiadou, S., and Rong, Y. Transformer for graphs: An overview from architecture perspective. *arXiv preprint arXiv:2202.08455*, 2022.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., and Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In Balcan, M. F. and Weinberger, K. Q. (eds.), *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pp. 1928–1937, New York, New York, USA, 20–22 Jun 2016. PMLR.
- Nair, V., Bartunov, S., Gimeno, F., Von Glehn, I., Lichocki, P., Lobov, I., O’Donoghue, B., Sonnerat, N., Tjandraatmadja, C., Wang, P., et al. Solving mixed integer programs using neural networks. *arXiv preprint arXiv:2012.13349*, 2020.
- Pace, R. K. and Barry, R. Sparse spatial autoregressions. *Statistics & Probability Letters*, 33(3):291–297, 1997.
- Papageorgiou, D. J., Nemhauser, G. L., Sokol, J., Cheon, M.-S., and Keha, A. B. Mirplib—a library of maritime inventory routing problem instances: Survey, core model, and benchmark results. *European Journal of Operational Research*, 235(2):350–366, 2014.
- Paulus, M. and Krause, A. Learning to dive in branch and bound. *Advances in Neural Information Processing Systems*, 36:34260–34277, 2023.
- Paulus, M. B., Zarpellon, G., Krause, A., Charlin, L., and Maddison, C. Learning to cut by looking ahead: Cutting plane selection via imitation learning. In *International conference on machine learning*, pp. 17584–17600. PMLR, 2022.
- Qi, M., Wang, M., and Shen, Z.-J. Smart feasibility pump: Reinforcement learning for (mixed) integer programming. *Reinforcement Learning for Real Life Workshop, International Conference on Machine Learning (ICML)*, 2021.
- Rong, Y., Bian, Y., Xu, T., Xie, W., Wei, Y., Huang, W., and Huang, J. Self-supervised graph transformer on large-scale molecular data. *Advances in neural information processing systems*, 33:12559–12571, 2020.
- Shaw, P. Using constraint programming and local search methods to solve vehicle routing problems. In *International conference on principles and practice of constraint programming*, pp. 417–431. Springer, 1998.
- Shen, Y., Sun, Y., Eberhard, A., and Li, X. Learning primal heuristics for mixed integer programs. In *2021 international joint conference on neural networks (ijcnn)*, pp. 1–8. IEEE, 2021.
- Shoja, S. and Axehill, D. Exact complexity certification of start heuristics in branch-and-bound methods for mixed-integer linear programming. In *2023 62nd IEEE Conference on Decision and Control (CDC)*, pp. 2292–2299. IEEE, 2023.
- Son, J., Kim, M., Kim, H., and Park, J. Meta-sage: Scale meta-learning scheduled adaptation with guided exploration for mitigating scale shift on combinatorial optimization. In *International Conference on Machine Learning*, pp. 32194–32210. PMLR, 2023.
- Son, J., Kim, M., Choi, S., Kim, H., and Park, J. Equity-transformer: Solving np-hard min-max routing problems as sequential generation with equity context. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 20265–20273, 2024.
- Song, J., Yue, Y., Dilkina, B., et al. A general large neighborhood search framework for solving integer linear programs. *Advances in Neural Information Processing Systems*, 33:20012–20023, 2020.
- Sonnerat, N., Wang, P., Ktena, I., Bartunov, S., and Nair, V. Learning a large neighborhood search algorithm for mixed integer programs. *arXiv preprint arXiv:2107.10201*, 2021.
- Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

- Tang, Y., Agrawal, S., and Faenza, Y. Reinforcement learning for integer programming: Learning to cut. In *International conference on machine learning*, pp. 9367–9376. PMLR, 2020.
- Tomlin, J. A. *On scaling linear programming problems*. Springer, 1975.
- Toth, P. and Vigo, D. *The vehicle routing problem*. SIAM, 2002.
- Tsourakakis, C., Gkantsidis, C., Radunovic, B., and Vojnovic, M. Fennel: Streaming graph partitioning for massive scale graphs. In *Proceedings of the 7th ACM international conference on Web search and data mining*, pp. 333–342, 2014.
- Vanderbei, R. J. Linear programming: foundations and extensions. *Journal of the Operational Research Society*, 49(1):94–94, 1998.
- Wu, X., Chen, Z., Wang, W. W., and Jadbabaie, A. A non-asymptotic analysis of oversmoothing in graph neural networks. In *The Eleventh International Conference on Learning Representations*, 2023.
- Wu, Y., Song, W., Cao, Z., and Zhang, J. Learning large neighborhood search policy for integer programming. *Advances in Neural Information Processing Systems*, 34: 30075–30087, 2021a.
- Wu, Z., Jain, P., Wright, M., Mirhoseini, A., Gonzalez, J. E., and Stoica, I. Representing long-range context for graph neural networks with global attention. *Advances in Neural Information Processing Systems*, 34:13266–13279, 2021b.
- Ye, H., Xu, H., Wang, H., Wang, C., and Jiang, Y. Gnn&gbdt-guided fast optimizing framework for large-scale integer programming. In *International conference on machine learning*, pp. 39864–39878. PMLR, 2023.
- Ye, H., Xu, H., and Wang, H. Light-milpopt: Solving large-scale mixed integer linear programs with lightweight optimizer and small-scale training dataset. In *The Twelfth International Conference on Learning Representations*, 2024.
- Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., and Liu, T.-Y. Do transformers really perform badly for graph representation? *Advances in neural information processing systems*, 34:28877–28888, 2021.
- Yoon, T. Confidence threshold neural diving. *NeurIPS 2021 MLACO competition track*, 2022.
- Zeng, H., Wang, J., Das, A., He, J., Han, K., Hu, H., and Sun, M. Effective generation of feasible solutions for integer programming via guided diffusion. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 4107–4118, 2024.
- Zhang, J., Zhang, H., Xia, C., and Sun, L. Graph-bert: Only attention is needed for learning graph representations. *arXiv preprint arXiv:2001.05140*, 2020.
- Zong, Z., Zheng, M., Li, Y., and Jin, D. Mapdp: Cooperative multi-agent reinforcement learning to solve pickup and delivery problems. In *Proceedings of the AAAI conference on artificial intelligence*, volume 36, pp. 9980–9988, 2022.
- Zuo, Y., Tharmarasa, R., Jassemi-Zargani, R., Kashyap, N., Thiyaalingam, J., and Kirubarajan, T. T. Milp formulation for aircraft path planning in persistent surveillance. *IEEE Transactions on Aerospace and Electronic Systems*, 56(5):3796–3811, 2020.

A. Proof of Proposition 1

Proposition 1 establishes the theoretical soundness of the reward formulation in achieving feasibility. In this section, we provide the proof of the proposition: *Suppose $\mathbf{x}_t \notin \mathcal{F}$, $\mathcal{R}_{t,\text{const}} > 0$, and $\mathcal{R}_{t,\text{bound}} = 0$ for all $t < \mathcal{T}$. Then $\mathbf{x}_{\mathcal{T}} \in \mathcal{F}$.*

A.1. Term Definitions

- $\mathcal{F}$: The set of feasible solutions (feasible region) for the given original ILP problem.
- $\mathbf{x}_t$: The solution of the problem at timestep t .
- $\mathcal{R}_{t,\text{bound}}$: The bound reward at timestep t (see Eq. 7).
- $\mathcal{R}_{t,\text{const}}$: The constraint reward at timestep t (see Eq. 8).
- $a_{j,i}$: The coefficient of the i -th variable in the j -th linear constraint.
- $x_{t,i}$: The value of the i -th variable at timestep t .
- $f_{t,j}$: The feasibility state for the j -th linear constraint at timestep t .
- $lhs_{t,j}$: The left-hand side of the j -th linear constraint at timestep t .
- b_j : The right-hand side of the j -th linear constraint.

A.2. Preliminaries

In our system, the action policy ensures that the integrality requirements (Eq. 1c) are satisfied (see Section 3.1.1). Therefore, integrality constraints can be omitted from this analysis. Furthermore, since *phase1* continues until the first feasible solution is found, we focus on the satisfaction of linear constraints (Eq. 1b) and bound constraints (Eq. 1d) during *phase1*.

Since a well-trained agent selects actions that maximize the reward, the rewards $\mathcal{R}_{t,\text{const}}$ and $\mathcal{R}_{t,\text{bound}}$ are expected to be positive and zero, respectively, following the definitions in Eqs. 7 and 8. Accordingly, we suppose that $\mathcal{R}_{t,\text{const}} > 0$ and $\mathcal{R}_{t,\text{bound}} = 0$ during *phase1* (i.e., exploration starting from an infeasible solution) in order to analyze the alignment between our reward design and the objective of achieving feasibility in ILP.

A.3. Proof

Suppose $\mathbf{x}_t \notin \mathcal{F}$, $\mathcal{R}_{t,\text{const}} > 0$, and $\mathcal{R}_{t,\text{bound}} = 0$. By the definition of the constraint reward, we have:

$$\mathcal{R}_{t,\text{const}} = \sum_{j=1}^m \min(f_{t+1,j}, 0) - \min(f_{t,j}, 0) > 0$$

In the context of ILP, the space of variables x_i is discrete. Since $\mathbf{A}$ and $\mathbf{b}$ consist of constant coefficients, the value of $lhs_{t,j} (= \sum_{i=1}^n a_{j,i} x_{t,i})$ is restricted to a discrete set. Consequently, $f_{t,j} (= b_j - lhs_{t,j})$ also changes in discrete increments. This discreteness implies that $\mathcal{R}_{t,\text{const}}$ must be at least a fixed minimum value δ whenever $\mathbf{x}_t \notin \mathcal{F}$. Thus, we have:

$$\mathcal{R}_{t,\text{const}} = \sum_{j=1}^m \min(f_{t+1,j}, 0) - \min(f_{t,j}, 0) \geq \delta > 0.$$

Let $D_t = \sum_{j=1}^m \min(f_{t,j}, 0)$ at step t . From this, we recursively obtain the general form:

$$D_1 \geq D_0 + \delta$$

$$D_2 \geq D_1 + \delta \geq D_0 + 2\delta$$

$$\vdots$$

$$D_t \geq D_0 + t \cdot \delta$$

As the initial solution is infeasible, it follows that $D_0 < 0$. Given that $\delta > 0$ by the definition above, there must exist a finite timestep t such that $D_0 + t \cdot \delta \geq 0$. Solving this inequality for t yields $t \geq -D_0/\delta$. Let $\mathcal{T} = \lceil -D_0/\delta \rceil$ be the smallest

timestep satisfying this condition. Given that $D_t \leq 0$ inherently due to the $\min(\cdot)$ function, $D_{\mathcal{T}} \geq 0$ forces $D_{\mathcal{T}} = 0$. Thus, $D_{\mathcal{T}} = \sum_{j=1}^m \min(f_{\mathcal{T},j}, 0) = 0$.

Given the nature of the $\min(\cdot)$ function, $f_{\mathcal{T},j}$ must be greater than or equal to 0 for all j , which implies that the linear constraints (Eq. 1b) are satisfied as follows:

$$f_{\mathcal{T},j} = b_j - \text{lhs}_{\mathcal{T},j} \geq 0, \forall j$$

$$\text{lhs}_{\mathcal{T},j} \leq b_j, \forall j$$

$$\mathbf{A}\mathbf{x}_{\mathcal{T}} \leq \mathbf{b}$$

Furthermore, regarding $\mathcal{R}_{t,\text{bound}}$, a value of zero indicates that no decision variables violate their bounds (see Eq. 7). Combined with the integrality guarantee outlined in Appendix A.2, all constraints are fully satisfied. Hence, $\mathbf{x}_{\mathcal{T}} \in \mathcal{F}$ (i.e., $\mathbf{x}_{\mathcal{T}}$ is feasible).

B. Pseudo-Code for Feasibility-Aware Variable Selection

Algorithm 2 Feasibility-aware variable selection

Input: instance M , observation $\mathcal{S}_t = (\mathbf{x}_t, \mathbf{f}_t, \text{obj}_t)$, current phase phase

Parameter: number of seed variables p , number of neighbor variables q

Output: observation with selected variables $\tilde{\mathcal{S}}_t = (\tilde{\mathbf{x}}_t, \tilde{\mathbf{f}}_t, \text{obj}_t)$

```

1:  $\tilde{\mathbf{A}} \leftarrow \mathbb{I}(\mathbf{A}_{j,i} \neq 0)_{j=1,\dots,m; i=1,\dots,n}$ 
2: if  $\text{phase} = 1$  then
3:    $\tilde{\mathbf{f}}_t \leftarrow \mathbb{I}(f_{t,j} < 0)_{j=1,\dots,m}$ 
4:    $\text{score\_seed} \leftarrow \tilde{\mathbf{f}}_t^\top \tilde{\mathbf{A}}$   $\{\tilde{\mathbf{f}}_t \in \mathbb{R}^{m \times 1}, \tilde{\mathbf{A}} \in \mathbb{R}^{m \times n}\}$ 
5:    $\text{weight} \leftarrow (\max(\text{abs}(\mathbf{c}^\top)) - \text{abs}(\mathbf{c}^\top) + 1) / \max(\text{abs}(\mathbf{c}^\top))$ 
6: else if  $\text{phase} = 2$  then
7:    $\tilde{\mathbf{f}}_t \leftarrow \mathbb{I}(f_{t,j} > 0)_{j=1,\dots,m}$ 
8:    $\text{score\_seed} \leftarrow \tilde{\mathbf{f}}_t^\top \tilde{\mathbf{A}}$   $\{\tilde{\mathbf{f}}_t \in \mathbb{R}^{m \times 1}, \tilde{\mathbf{A}} \in \mathbb{R}^{m \times n}\}$ 
9:    $\text{score\_seed} \leftarrow \max(\text{score\_seed}) - \text{score\_seed} + 1$ 
10:   $\text{weight} \leftarrow \text{abs}(\mathbf{c}^\top) / \max(\text{abs}(\mathbf{c}^\top))$ 
11: end if
12:  $\text{score\_seed} \leftarrow \text{score\_seed} \odot \text{weight}$   $\{\text{score\_seed} \in \mathbb{R}^{1 \times n}\}$ 
13:  $\text{prob} \leftarrow \text{score\_seed} / \text{sum}(\text{score\_seed})$   $\{\text{prob} \in \mathbb{R}^{1 \times n}\}$ 
14:  $\text{indices\_seed} \leftarrow \text{sample}(\text{prob}, p)$  {Sample  $p$  seed variables according to prob}
15:  $\mathbf{g} \leftarrow \text{rowwise\_sum}(\tilde{\mathbf{A}}[:, \text{indices\_seed}])$   $\{\mathbf{g} \in \mathbb{R}^{m \times 1}\}$ 
16:  $\text{score\_neighbor} \leftarrow \mathbf{g}^\top \tilde{\mathbf{A}}$   $\{\text{score\_neighbor} \in \mathbb{R}^{1 \times n}\}$ 
17:  $\text{score\_neighbor}[:, \text{indices\_seed}] \leftarrow -1$  {Prevent to select seed variables}
18:  $\text{indices\_neighbor} \leftarrow \text{top}(\text{score\_neighbor}, q)$  {Obtain top  $q$  neighbor variables}
19:  $\text{changeable} \leftarrow \text{concatenate}(\text{indices\_seed}, \text{indices\_neighbor})$ 
20:  $\tilde{\mathbf{x}}_t \leftarrow \mathbf{x}_t[\text{changeable}]$  {Select  $\tilde{n}(= p + q)$  variables}
21:  $\tilde{\mathcal{S}}_t \leftarrow (\tilde{\mathbf{x}}_t, \tilde{\mathbf{f}}_t, \text{obj}_t)$ 
22: return  $\tilde{\mathcal{S}}_t$ 

```

C. Pseudo-Code for Learning Algorithm

We obtain the initial solution (Line 2) through one of two methods: LP-relaxation or random assignment. For LP-relaxation, we apply random rounding to the LP-feasible solution to convert the fractional variable values into integers, which may result in an infeasible solution for the original ILP problem. For random assignment, when training on the first instance, we randomly assign the value 1 to 1% of the variables. For subsequent instances, we assign the value 1 to r randomly selected variables, where r is half the number of variables with non-zero values in the previous instance.

Algorithm 3 Learning a policy for RL-SPH

Input: agent parameters θ , instance M

Parameter: update limit N , total step limit T_{max} , step limit for *phase1* T_{stay}

Output: updated parameters θ

```

1: for  $N$  updates do
2:    $\mathbf{x}_0 \leftarrow \text{get\_initial\_solution}(M)$ 
3:    $\mathcal{S}_0 \leftarrow \text{observe}(M, \mathbf{x}_0)$ 
4:    $\mathbf{x}_b \leftarrow \emptyset$ 
5:    $obj_b \leftarrow \infty$ 
6:    $\text{phase}_0 \leftarrow 1$ 
7:    $\text{stay} \leftarrow \text{True}$ 
8:   for  $t = 0, 1, 2, \dots, T_{max}$  do
9:      $\mathcal{R}_{t,\text{total}}, \mathcal{S}_{t+1}, \mathbf{x}_b, obj_b \leftarrow \text{search}(M, \pi_\theta, \mathcal{S}_t, \mathbf{x}_b, obj_b, \text{phase}_t)$  {See Algorithm 1}
10:    if  $\text{stay} = \text{True}$  and  $(\mathbf{x}_{t+1} \in \mathcal{F} \text{ or } t = T_{stay})$  then
11:      if  $t = T_{stay}$  then
12:         $\text{stay} \leftarrow \text{False}$ 
13:      end if
14:       $\mathcal{S}_{t+1} \leftarrow \mathcal{S}_0$ 
15:       $\mathbf{x}_b \leftarrow \emptyset$ 
16:       $obj_b \leftarrow \infty$ 
17:    else if  $\text{stay} = \text{False}$  and  $\mathbf{x}_{t+1} \in \mathcal{F}$  then
18:       $\text{phase}_{t+1} \leftarrow 2$ 
19:    end if
20:     $\delta_{td} \leftarrow \mathcal{R}_{t,\text{total}} + \gamma \cdot V_\theta(\mathcal{S}_{t+1}, \text{phase}_{t+1}) - V_\theta(\mathcal{S}_t, \text{phase}_t)$ 
21:     $\mathcal{L}_\theta \leftarrow -\log \pi_\theta(\mathcal{A}_t \mid \mathcal{S}_t, \text{phase}_t) \cdot \delta_{td} + \delta_{td}^2$ 
22:     $\theta \leftarrow \text{update}(\mathcal{L}_\theta, \theta)$ 
23:  end for
24: end for
25: return  $\theta$ 

```

D. Additional Preliminaries

D.1. Properties of ILP

All ILP problems can be transformed into the standard form (Eqs. 1a–1d) (Bertsimas & Tsitsiklis, 1997). Let $\mathbf{a}_i^\top$ denote a row vector of a constraint, $\mathbf{A} = (\mathbf{a}_1^\top, \dots, \mathbf{a}_m^\top)$, and $\mathbf{b} = (b_1, \dots, b_m)$. An equality constraint $\mathbf{a}_i^\top \mathbf{x} = b_i$ is equivalent to two inequality constraints: $\mathbf{a}_i^\top \mathbf{x} \geq b_i$ and $\mathbf{a}_i^\top \mathbf{x} \leq b_i$. Moreover, $\mathbf{a}_i^\top \mathbf{x} \geq b_i$ is equivalent to $-\mathbf{a}_i^\top \mathbf{x} \leq -b_i$. Maximizing $\mathbf{c}^\top \mathbf{x}$ is equivalent to minimizing $-\mathbf{c}^\top \mathbf{x}$. Thus, we only address the standard form (i.e., minimization) in this study.

ILP is known to be NP-hard due to its integrality constraints (Eq. 1c) (Berthold, 2006). As the number of integer variables increases, the computational cost grows exponentially (Floudas, 1995). LP-relaxation transforms an original ILP problem into an LP one by removing the integrality constraints (Bertsimas & Tsitsiklis, 1997). Although LPs are computationally cheaper to solve, which can be solved in polynomial time (Karmarkar, 1984; Vanderbei, 1998), an LP-feasible solution may be infeasible for the original ILP due to the integrality constraints (Eq. 1c) (Guieu & Chinneck, 1999).

D.2. Start Primal Heuristics for ILP

Start primal heuristics (SPH) do not require an initial ILP-feasible solution. Instead, they typically begin with an LP-feasible solution and attempt to convert it into an ILP-feasible one (Berthold, 2006), shown in Figure 1(b). Representative methods include diving heuristics, feasibility pump (FP), rounding heuristics, and relaxation enforced neighborhood

search (RENS) (Berthold, 2006; Shoja & Axehill, 2023). Specifically, diving heuristics fix fractional variables in the LP solution to promising integer values and iteratively resolve the LP. FP alternates between two sequences, one LP-feasible and the other ILP-feasible, with the goal of convergence to a feasible ILP solution. Rounding heuristics attempt to obtain an ILP-feasible solution by rounding fractional LP values up or down. RENS constructs and solves a sub-ILP of the original problem by fixing or tightening bounds of integer variables based on the LP solution.

D.3. Bipartite Graph Representation of ILP

Prior studies on E2EPH represent ILP instances as bipartite graphs (Nair et al., 2020; Yoon, 2022; Han et al., 2023; Cantürk et al., 2024; Huang et al., 2024; Liu et al., 2025a). In this representation, one set of nodes corresponds to constraints, and the other to decision variables. An edge connects a variable node to a constraint node if and only if the variable appears in the corresponding constraint. For example, in Figure 2(a), variable x_3 appears in constraint a_2 ; thus, the node representing x_3 is connected to the node representing a_2 in the bipartite graph.

D.4. Graph Neural Networks for ILP

Existing E2EPH methods are typically built upon the message passing neural network (MPNN) framework (Gilmer et al., 2017). MPNNs aggregate messages from the neighbor nodes, making them well-suited for capturing local structural information. However, they struggle to capture long-range dependencies between distant nodes (Zhang et al., 2020; Wu et al., 2021b). To propagate messages between nodes that are K hops apart, an MPNN requires at least K layers.

In the context of ILP, capturing relationships among variables that influence each other across multiple constraints may require deep MPNNs. However, deeper architecture often suffers from the oversmoothing problem (Li et al., 2018; Wu et al., 2021b; Min et al., 2022). For instance, as shown in Figure 2(a), variables x_2 and x_3 are four hops apart: $x_2 - a_1 - x_1 - a_2 - x_3$. Although x_2 and x_3 do not appear in the same constraint, they are connected via x_1 , which is shared by both a_1 and a_2 . A change in x_2 can affect x_1 , which in turn may influence x_3 . Modeling such interactions would require four layers, but even shallow MPNNs with 2–4 layers are prone to oversmoothing (Wu et al., 2023). Thus, we design a new Graph Neural Network (GNN) for ILP inspired by Graph Transformers (Rong et al., 2020; Ying et al., 2021; Lin et al., 2022; Min et al., 2022), which can effectively learn relationships between distant nodes (Wu et al., 2021b).

E. Related Work

E.1. Machine Learning for ILP

ML techniques for solving ILP can be broadly categorized into three groups (Bengio et al., 2021). The first group, *learning to configure algorithms*, uses ML to optimize the configuration of specific components within ILP solvers. Examples include deciding parameters for promising configurations (Hutter et al., 2011), applying decomposition (Kruber et al., 2017), and selecting scaling methods (Berthold & Hendel, 2021). The second group, *ML alongside optimization algorithms*, integrates ML into ILP solvers to aid key decisions during the optimization, such as cut selection (Tang et al., 2020; Paulus et al., 2022), variable selection (Khalil et al., 2016; Alvarez et al., 2017; Gasse et al., 2019; Gupta et al., 2020; Paulus & Krause, 2023), node selection in the branch-and-bound (He et al., 2014; Labassi et al., 2022), and neighborhood selection in LNS (Song et al., 2020; Sonnerat et al., 2021; Wu et al., 2021a; Huang et al., 2023; Liu et al., 2025b).

RL-SPH belongs to the third group, *end-to-end learning*, which uses ML to directly learn and predict solutions. This group includes existing E2EPH methods (Nair et al., 2020; Shen et al., 2021; Yoon, 2022; Han et al., 2023; Cantürk et al., 2024; Huang et al., 2024; Zeng et al., 2024; Liu et al., 2025a; Geng et al., 2025) as well. PAS (Han et al., 2023) adopts a predefined trust region instead of strictly fixing variables, which can be viewed as the generalization of the fixing strategy (Huang et al., 2024). Recent methods (Huang et al., 2024; Liu et al., 2025a) follow this approach as well. Although using trust regions alleviates the risk of infeasibility, these methods still rely on ILP solvers to obtain feasible solutions. Several efforts have been made to solve ILPs via ML without relying on ILP solvers (Zeng et al., 2024; Geng et al., 2025), but none have attained a 100% feasibility rate. Specifically, Zeng et al. reported that even when their proposed method is combined with SCIP’s heuristic, it still fails to achieve 100% feasibility on the CA benchmark. Similarly, Geng et al. documented the average feasible ratios of 50.8%, 97.1%, and 99.4% on the SC, IS, and CA benchmarks, respectively; furthermore, they observed that PAS (Han et al., 2023) and ConPaS (Huang et al., 2024) fail to find feasible solutions for the majority of instances. Moreover, the aforementioned methods primarily focus on binary variables. To the best of our knowledge, RL-SPH is the first standalone E2EPH method that learns to generate feasible solutions independently, establishing a theoretical reward-feasibility alignment and empirically achieving a 100% feasibility rate across the five evaluated benchmarks, even for ILP involving non-binary integers.

E.2. Positioning Relative to Prior Work

RL-SPH is a *start primal heuristic* designed to quickly achieve feasibility from ILP-infeasible solutions. As discussed in Appendix E.1, RL-SPH can also be categorized under *end-to-end learning*, but it fundamentally differs from existing approaches in terms of how feasible solutions are obtained. For example, as illustrated in Figure 1, typical E2EPH methods rapidly approximate a solution and then delegate the remaining optimization—including ensuring feasibility—to an external solver. In this workflow, the solver bears responsibility for producing feasible solutions, typically via its built-in start primal heuristics. Consequently, most prior E2EPH work has evolved under the assumption that feasibility will be handled by the external solver rather than by the learned model itself.

This distinction underscores that RL-SPH addresses a more challenging problem than existing E2EPH work, as it must generate feasible solutions *without* relying on external solvers. Many real-world applications—such as real-time risk assessment (Chen et al., 2023)—require high-quality feasible solutions quickly and repeatedly (Zeng et al., 2024; Cantürk et al., 2024). In such settings, RL-SPH is advantageous because it can quickly deliver feasible solutions independently of a solver. Notably, the LP-free variant of RL-SPH discovers the first feasible solution $180\times$ faster than established SPH baselines on average (see Section 4.2).

Feasibility is a necessary prerequisite for any subsequent optimization stage. For example, widely used ILP metaheuristics such as local branching (Fischetti & Lodi, 2003; Liu et al., 2022) and large neighborhood search (LNS) (Shaw, 1998; Song et al., 2020; Wu et al., 2021a; Huang et al., 2023; Liu et al., 2025b) require an initial feasible solution to operate and further improve the incumbent solution. As a result, infeasibility constitutes a significant bottleneck in solving ILP problems. However, finding a feasible solution remains inherently challenging due to the integrality constraints (Eq. 1c), which induce an exponential increase in computational cost with respect to the number of integer variables (Floudas, 1995). Moreover, even well-established primal heuristics typically cannot guarantee feasibility (Shoja & Axehill, 2023). Thus, the ability to reliably generate feasible solutions for general ILPs is crucial. Despite the importance of feasibility, learning-based approaches for directly addressing this problem remain underexplored. Advancing learning-based methods for achieving feasibility establishes a crucial foundation for ML-based standalone solvers pursuing optimality, especially given that achieving optimality is recognized as a major challenge in the ML for CO community (Son et al., 2023).

Existing E2EPH methods and RL-SPH possess *orthogonal* advantages, making them ideal candidates for a hybrid approach. Specifically, RL-SPH can leverage existing E2EPH as a solution initializer, a synergy demonstrated in Section 4.3. RL-SPH resembles LNS in that it explores neighborhoods larger than those used in traditional local search (see Section 2.3). However, as previously noted, existing learning-based LNS methods require an initially feasible solution and aim to improve the incumbent solution, which positions them in a different stage of the solution process from start primal heuristics. Therefore, they are not direct points of comparison for our study. Nonetheless, insights from the LNS literature could be leveraged to further enhance the behavior of RL-SPH *after* it obtains a feasible solution.

E.3. Evolution of the Neural Local Rewriting Paradigm

NeuRewriter (Chen & Tian, 2019) is a pioneering method that employs an actor-critic algorithm for rewriting solutions in CO tasks such as job scheduling and vehicle routing. While RL-SPH shares the conceptual similarities of identifying and “rewriting” sub-regions of a solution, it represents a significant progression in two key dimensions:

1. **From infeasible to feasible:** NeuRewriter is suited for problems where an initial feasible solution is readily available, focusing on improving the incumbent solution (aligning with our *phase2*). Chen & Tian (2019) envisioned extending this approach to a broader range of problems that are hard for classic algorithms when starting with an infeasible solution. RL-SPH directly realizes this vision by establishing a concrete framework to navigate from infeasible to feasible solutions (*phase1*), addressing the previously identified challenge.
2. **General ILP formulation:** NeuRewriter relies on task-specific architectures (e.g., dedicated encoders). In contrast, RL-SPH operates on an ILP formulation, serving as a general-purpose framework. We validated this with a consistent architecture and hyperparameter configuration across five benchmarks. While we do not claim RL-SPH is optimal for every specific problem, we emphasize its versatility—it can be applied to various CO problems expressible as an ILP without requiring significant architectural adjustments.

This comparison contextualizes the paradigm’s evolution, highlighting RL-SPH’s contribution in bridging the gap between infeasibility and feasibility within a general-purpose framework.

F. Additional Explanation of Reward System

F.1. Rationale on Reward Design

As defined in Section 2.1, ILP is an optimization problem that minimizes an objective value while satisfying a set of constraints: linear constraints, integrality constraints, and variable bounds. When framing ILP within an RL system, the reward mechanism must likewise be designed to reflect this definition. Accordingly, we derived the following fundamental types of rewards: those related to objective values, linear constraints, and variable bounds. The reward for integrality constraints is omitted because they are handled by the action policy (see Section 3.1.1).

According to the above definition, satisfying the set of constraints is a prerequisite for performing optimization. Therefore, rewards related to the set of constraints should take high priority over the objective-value reward. We believe it is intuitively reasonable that the more a bound or linear constraint is violated, the greater the penalty should be. Likewise, a larger reward is appropriate as the objective value improves. From an optimization perspective, improving the feasibility of linear constraints while violating variable bounds is meaningless. For example, in an ILP where variables must take values of either 0 or 1, assigning a value such as -1 is invalid. Thus, satisfying the variable bounds has the highest priority among the variable bounds, linear constraints, and objective values. The reward structure described in Eq. 4 obeys the mentioned priority. Additionally, to avoid the bound penalty being overly dominated by the constraint reward, we scaled it by the square root of the number of changeable variables. The maximum value of the objective coefficients scales the reward regarding the objective value. This scaling helps stabilize learning by mitigating large reward fluctuations as well.

F.2. Feasibility Reward

The feasibility reward $\mathcal{R}_{t,F}$ is calculated in proportion to the degree of feasibility improvement or deterioration. In Figure 7, the agent violates Constraint-1 while satisfying Constraint-2. If the agent moves closer to the feasible region, it is considered an improvement in feasibility. Conversely, if it moves further away and ends up violating Constraint-2 as well, it is regarded as a deterioration. In this way, rewards are assigned based on how much the agent’s actions improve or worsen feasibility. As a result, the agent learns to find feasible solutions to maximize its rewards.

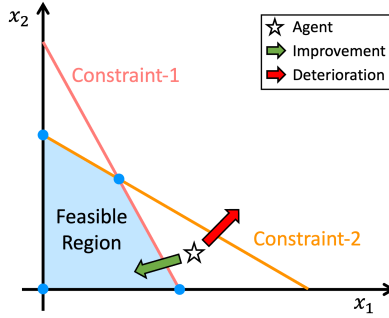


Figure 7. Illustration of feasibility improvement and deterioration.

F.3. Reward in *phase2*

The green circles in Figure 8 represent feasible solutions whose objective values are $obj_{t+1} < obj_b$ (Case 1 in Eq. 5). We regard the agent’s actions that fail to find better feasible solutions than $\mathbf{x}_b$ as incorrect (Cases 2, 3, 4). The red circle indicates a feasible solution worse than the incumbent (Case 2), while all triangles correspond to infeasible solutions (Cases 3, 4).

Let the objective values be $obj_b = -10$, $\mathbf{q}_1 = -16$, $\mathbf{q}_2 = -13$, $\mathbf{q}_3 = -11$, $\mathbf{q}_4 = -8$, and $\mathbf{q}_5 = -7$, with $\alpha = 2$ and $\max(|\mathbf{c}|) = 1$. For the green circles, rewards in *phase2* are calculated by the first case in Eq. 5. The rewards for $\mathbf{q}_1$ and $\mathbf{q}_3$ are $\Delta obj_t = \frac{|-16 - (-10)|}{1} = 6$ and $\Delta obj_t = \frac{|-11 - (-10)|}{1} = 1$, respectively. Since $\mathbf{q}_1$ has the better objective value than $\mathbf{q}_3$, it receives a higher reward. For the red circle, the reward is $-\Delta obj_t \cdot \alpha = -\frac{|-8 - (-10)|}{1} \cdot 2 = -4$. For the triangles, the penalties for $\mathbf{q}_2$ and $\mathbf{q}_5$ are $\mathcal{R}_{t,F}$ and $\mathcal{R}_{t,F} \cdot \alpha = 2 \cdot \mathcal{R}_{t,F}$, respectively. The distances from obj_b to $\mathbf{q}_2$ and $\mathbf{q}_5$ are the same (i.e., $|-13 - (-10)| = |-7 - (-10)| = 3$), but the penalty for $\mathbf{q}_2$ is smaller than that for $\mathbf{q}_5$ due to amplifying by α .

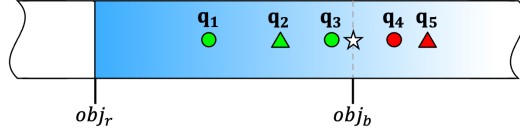


Figure 8. Illustration of reward function in *phase2*. $\circ$: feasible, $\triangle$: infeasible, green: inside, red: outside, $\star$: agent, obj_r : the objective value of the LP-feasible solution, obj_b : incumbent value.

F.4. Toward-Optimal Bias

Figure 9 visualizes the agent’s potential penalties by $\mathcal{R}_{t,F}$ in *phase2*. The third and fourth quadrants illustrate the penalties, assuming that $\mathcal{R}_{t,F}$ is a linear function of the gap from obj_{t+1} to the incumbent value obj_b for simplicity. As illustrated in Figures 9(a) and 9(b), a higher α results in higher penalties for solutions with $obj_{t+1} > obj_b$. By controlling the toward-optimal bias α , we can guide the agent to explore promising regions (i.e., $obj_{t+1} < obj_b$) for better feasible solutions rather than making ineffective moves (i.e., $obj_{t+1} > obj_b$), thus increasing its opportunities for learning. Appendix H.4.1 provides experimental results on the toward-optimal bias α .

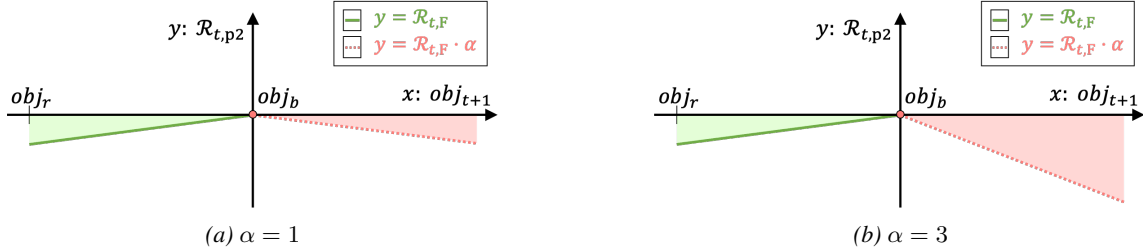


Figure 9. Illustration of the potential penalties of $\mathcal{R}_{t,p2}$ as a function of the objective value obj_{t+1} in *phase2*. The x -axis represents obj_{t+1} , and the y -axis represents $\mathcal{R}_{t,p2}$, with the origin set at obj_b .

G. Details of Experimental Setup

G.1. Benchmark Datasets

Table 7 shows the average sizes of each benchmark dataset used in our experiments. We generated instances for IS, CA, SC, and MVC following the code¹ from (Gasse et al., 2019). For NBI, instances were generated based on the description in (Qi et al., 2021). Table 8 summarizes the parameters used for NBI instance generation. Considering that the ratio of non-zero coefficients ρ in typical LP problems is less than 5% (Hillier & Lieberman, 2015), we set a higher density of 10% to promote more interactions between variables in constraints. According to the default settings of the ILP solver Gurobi (Gurobi Optimization, 2025) and SCIP (Bestuzheva et al., 2021), the lower bound l_i and upper bound u_i for decision variables are set to 0 and ∞ , respectively.

Table 7. Average sizes of each dataset.

Dataset	# binary variables	# integer variables	# constraints	Density
Independent set (IS)	1,500	0	5,941	0.13%
Combinatorial auction (CA)	4,000	0	2,677	0.21%
Set covering (SC)	3,000	0	2,000	5%
Minimum vertex cover (MVC)	3,000	0	11,931	0.07%
Non-binary integers (NBI)	0	2,000	2,000	10%

NBI instances follow the structure of an Unbounded Multidimensional Knapsack Problem, with non-zero coefficients in $\mathbf{A}$ sampled from $\{1, \dots, 10\}$, positive RHS values, and $\mathbf{c}$ from $\{-10, \dots, -1\}$. These instances are formulated in the standard form as in Eq. 1: $\min. \mathbf{c}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{A}\mathbf{x} \leq \mathbf{b}, \quad x_i \in \mathbb{Z}_{\geq 0}, \forall i$. Under this formulation, $\mathbf{A}$ and $\mathbf{b}$ can be interpreted as representing resource consumption and capacities, respectively. Since all objective coefficients are negative, the minimization objective is equivalent to maximizing total profit (i.e., maximizing $-\mathbf{c}^\top \mathbf{x}$).

¹<https://github.com/ds4dm/learn2branch>

Table 8. Parameters for non-binary integer instance generation.

Parameter	Distribution
c	$\text{randint}[-10, -1]$
A	$\text{randint}[1, 10]$ with density $\rho = 0.1$
b	$\mathbf{A}\xi + \epsilon$, where $\xi_i \sim \text{randint}[1, 10], \forall i = 1, \dots, n$ and $\epsilon_j \sim \text{randint}[1, 10], \forall j = 1, \dots, m$
l_i	$0, \forall i = 1, \dots, n$
u_i	$\infty, \forall i = 1, \dots, n$

G.2. Implementation Details

The following hyperparameter configuration remained fixed across all experiments, except for the sensitivity analyses in Appendix H.4. RL-SPH was trained concurrently on 64 distinct instances, which serves as the batch size of ILP-GT. We maintained a consistent configuration for Eq. 5, Algorithm 2, and Algorithm 3: the toward-optimal bias $\alpha = 2$, the number of seed/neighbor variables $p = q = \lceil \log_2 n \rceil$, the update limit $N = 5000$, the total step limit $T_{max} = 2000$, and the step limit for *phase1* $T_{stay} = 500$. Training was conducted using Algorithm 3 with the RMSprop optimizer (learning rate $= 1 \times 10^{-4}$, epsilon $= 1 \times 10^{-5}$, alpha $= 0.99$, weight decay $= 1 \times 10^{-3}$), where the learning rate was linearly decayed.

We implemented our ILP-GT based on the Graph Transformer code from GitHub² (Wu et al., 2021b), with the same configuration. We utilized the official implementation³ of the positional encoding module (Gorishniy et al., 2022). Our RL algorithm is built upon the Actor-Critic implementation in PyTorch⁴ (Kostrikov, 2018), modified to be tailored for ILP.

As shown in Table 9, the training durations for IS, CA, SC, MVC, and NBI were 31, 32, 26, 83, and 23 minutes, respectively. Notably, the training process is highly efficient, with all tested datasets requiring less than 90 minutes. Appendix H.5.3 also illustrates the stability and convergence of the training process. Table 9 also details the size of ILP-GT trained on each dataset. We compare RL-SPH’s training costs with existing E2EPH methods in Appendix I.2.

Table 9. Training time and model size of RL-SPH on each dataset described in Table 7.

Dataset	IS	CA	SC	MVC	NBI
Training time (min)	31	32	26	83	23
# trainable parameters	3.9M	2.7M	2.4M	6.2M	2.4M

G.3. Details of SPH Baselines

To ensure comprehensive coverage, DHF encompasses 15 diving heuristics, and RHF covers 6 rounding variants implemented in SCIP (v8.1.0), as follows:

- **Diving Heuristic Family (DHF):** *adaptivediving*, *conflictdiving*, *guideddiving*, *objpscostdiving*, *distributiondiving*, *farkasdiving*, *fracdiving*, *linesearchdiving*, *pscostdiving*, *veclendiving*, *rootsoldiving*, *nlpdiving*, *coefdiving*, *intdiving*, and *actconsdiving*.
- **Rounding Heuristic Family (RHF):** *simplerounding*, *rounding*, *zirounding*, *randrounding*, *shifting*, and *intshifting*.

G.4. Evaluation Environment

We conducted all evaluations under identical configurations. The evaluation machine is equipped with two AMD EPYC 7302 @ 3.0GHz, 2048GB RAM, and four NVIDIA A100 GPUs. All experiments were performed using a single NVIDIA A100 GPU. The software environment includes PyTorch 1.12.0, Gymnasium 0.29.1, and SCIP 8.1.0.

²<https://github.com/ucbrise/graphtrans>

³<https://github.com/yandex-research/rtdl-num-embeddings>

⁴<https://github.com/ikostrikov/pytorch-a2c-ppo-acktr-gail>

H. Additional Experiments

H.1. Further Comparison with SOTA E2EPH

We trained PAS, DDIM, and DiffILO using the default configurations and the authors’ released code^{5, 6, 7}. The best-performing PAS and DiffILO models were selected within 1,200 training epochs, and this epoch setting is among those used by Geng et al. (2025). As summarized in Table 10, PAS, DDIM, and DiffILO required approximately $8.7\times$, $1.4\times$, and $33.9\times$ more training time than RL-SPH, respectively. This indicates that the baselines were allocated sufficient training time in our experiment.

Table 10. Comparison of training duration (in minutes) and RL-SPH’s relative speedup against baseline methods.

Dataset	Training duration (min)				RL-SPH’s speedup		
	RL-SPH	PAS	DDIM	DiffILO	PAS	DDIM	DiffILO
SC	26	–	66	850	–	$2.5\times$	$32.7\times$
CA	32	343	43	1177	$10.7\times$	$1.3\times$	$36.8\times$
IS	31	209	14	1002	$6.7\times$	$0.5\times$	$32.3\times$
Average	29.7	276.0	41.0	1009.9	$8.7\times$	$1.4\times$	$33.9\times$

DiffILO achieved a 100% FR only on the IS dataset (see Table 2), whose characteristics closely match those used in the DiffILO study, as shown in Table 11. In contrast, SC differs in the number of variables and constraints, and our CA dataset contains approximately $2.7\times$ more variables and $4.7\times$ more constraints. This indicates that, even within the same problem class, variations in dataset characteristics may necessitate additional tuning for DiffILO to maintain high performance. This claim is further supported by Table 4 in the DiffILO paper, which shows that DiffILO performed dataset-specific tuning on 11 important hyperparameters. In contrast, RL-SPH did not employ any dataset-specific tuning; a consistent hyperparameter configuration was used for all datasets, as detailed in Appendix G.2. Therefore, the fact that RL-SPH achieved a 100% FR in all experiments without dataset-specific tuning demonstrates its robust capability to generate feasible solutions.

Table 11. Characteristics of datasets used in the DiffILO study and ours.

Dataset	Study	Number of variables	Number of constraints
SC	DiffILO	2,000	3,000
	Ours	3,000	2,000
CA	DiffILO	1,500	576
	Ours	4,000	2,677
IS	DiffILO	1,500	5,943
	Ours	1,500	5,941

H.2. Evaluation of Generalization

H.2.1. GENERALIZATION TO UNSEEN MIPLIB BENCHMARKS

We further evaluated the joint RL-SPH model on diverse instances from the MIPLIB benchmark (Gleixner et al., 2021) to assess its generalization capabilities. Table 12 presents summary statistics of the instances used in this experiment, along with incumbent values obtained by RHF and RL-SPH. Notably, the joint model obtains feasible solutions for these MIPLIB instances despite them differing substantially from the training instances (IS and MVC) in the following aspects:

1. **Variable domains:** rail507 and gen-ip002 contain continuous or integer variables with infinite upper bounds.
2. **Scale:** scpk4 contains nearly 67 times more variables than the training instances.
3. **Density:** Matrix density spans from highly sparse (0.1% in toll-like) to fully dense (100% in gen-ip016).

These results demonstrate that RL-SPH generalizes its problem-solving capabilities to unseen instances, even when encountering different variable types and domains.

⁵https://github.com/sribdcn/Predict-and-Search_MILP_method

⁶<https://github.com/MIRALab-USTC/L2O-DiffILO>

⁷<https://github.com/agent-lab/diffusion-integer-programming>

Table 12. Statistics of instances from the official MIPLIB website and the incumbent values of RL-SPH(IS+MVC) and RHF. To the best of our knowledge, the website does not provide information on how the *BKS* was obtained and how much computation time was used. The time limit for RHF and RL-SPH is set to 100 seconds.

Test instance	Variable				#constraints	Density	<i>BKS</i> ↓	Incumbent value	
	Total	#binaries	#integers	#continuous				RHF	RL-SPH
scpk4	100,000	100,000	0	0	2,000	0.5%	318	765	13,672
ex1010-pi	25,200	25,200	0	0	1,468	0.3%	233.0	706.0	660.0
toll-like	2,883	2,883	0	0	4,408	0.1%	610.0	1,314.0	1,774.0
f2gap401600	1,600	1,600	0	0	40	5.0%	82,307.0	82,307.0	82,307.0
seymour	1,372	1,372	0	0	4,944	0.5%	423.0	622.0	597.0
cod105	1,024	1,024	0	0	1,024	5.5%	-12.0	-2.0	-6.0
queens-30	900	900	0	0	960	10.8%	-40.0	-27.0	-24.0
rail507	63,019	63,009	0	10	509	1.5%	174	310	469
core4872-1529	24,656	24,645	0	11	4,875	0.2%	1,530.0	4,121.0	2,973.0
neos-1171737	2,340	1,170	0	1,170	4,179	0.6%	-195.0	-82.5	-101.0
neos-1442119	728	364	0	364	1,524	0.6%	-181.0	-122.4	-156.0
bppc8-02	232	229	1	2	59	32.0%	507	530	1,824
gen-ip002	41	0	41	0	24	93.7%	-4,783.7	-4,624.1	-4,678.2
gen-ip021	35	0	35	0	28	96.4%	2,361.5	2,486.7	2,507.7
gen-ip054	30	0	30	0	27	65.7%	6,841.0	7,148.7	7,061.0
gen-ip036	29	0	29	0	46	97.7%	-4,606.7	-4,504.2	-4,549.0
gen-ip016	28	0	28	0	24	100.0%	-9,476.2	-9,229.3	-9,352.3

In general, the joint model yielded incumbent values that are competitive with or superior to the strongest baseline, RHF. However, it struggled to sustain this performance on `scpk4`, where the number of variables differs significantly from the training set. Maintaining high performance on exceptionally large instances may require scaling up the training instances. Furthermore, the joint model lacks proficiency in handling specific constraints, such as Set Partitioning found in `bppc8-02`, which were absent during training. Future work should focus on specific constraint classes, extending RL-SPH’s capabilities beyond general constraints addressed in this work. Nevertheless, we emphasize that RL-SPH finds feasible solutions within an average of 12 seconds for these diverse MIPLIB instances, despite being jointly trained on IS and MVC which possess limited characteristics (e.g., binary-only variables and sparse matrices). This demonstrates RL-SPH’s potential to function as a unified solver across diverse problem settings, which is highly desirable in real-world CO (Liu et al., 2024).

H.2.2. DISTRIBUTION CHANGES

We conducted an experiment to evaluate the robustness of RL-SPH to changes in distribution. As described in Section 4.1.1, the IS dataset was generated using the Barabási–Albert random graph model (BA). BA generates scale-free networks that follow a power-law degree distribution. It captures the preferential attachment mechanism observed in many real-world networks, such as citation networks and social networks. In contrast, Erdős–Rényi random graph model (ER) constructs a graph by starting with u nodes and connecting each pair of nodes with probability p (Albert & Barabási, 2002). This process yields a graph where the expected number of edges is approximately $p \cdot u(u - 1)/2$, with edges distributed uniformly at random. Therefore, graphs generated by ER and BA have different degree distributions.

Table 13 shows that RL-SPH, trained on IS instances generated by BA, produced feasible solutions with a 100% FR on test instances generated by ER. Furthermore, RL-SPH outperforms RHF in terms of PG and PI, which is the strongest baseline, demonstrating that it robustly finds high-quality feasible solutions under distribution shifts.

Table 13. Generalization performance of RL-SPH under distribution changes.

Test dataset	Metric	RHF	RL-SPH trained on IS (BA)
IS (ER)	FR (%) ↑	100	100
	PG (%) ↓	7.02±2.39	0.00±0.00
	PI ↓	2.7±0.6	2.4±0.2

H.2.3. SIZE CHANGES

The application of ML to large-scale ILP has emerged as an important issue. Recently, there have been studies on ML-based heuristics for solving large-scale ILP (Ye et al., 2023; 2024). These works have proposed techniques that utilize a graph partitioning algorithm (Tsourakakis et al., 2014) to divide large-scale ILP instances into smaller subproblems for optimization. Their experimental results showed that breaking a large problem into smaller ones can effectively address the challenges of large-scale ILP. This finding suggests that, even if RL-SPH may not directly handle large-scale ILP, it has the potential to solve them by decomposing the original large-scale problem into smaller subproblems. Therefore, we believe the datasets used in our experiments are sufficient for evaluating our method. Nevertheless, to more thoroughly assess its robustness, we experimented on scalability, as follows.

Table 14 shows that RL-SPH trained on SC ($n = 3,000$) successfully generated feasible solutions for larger test instances. Specifically, RL-SPH achieved a 100% FR even on test instances with more than three times the number of variables as the training instances. RL-SPH consistently found higher-quality solutions than RHF, indicating that a model trained on smaller instances can generalize effectively to larger ones.

Table 14. Generalization performance of RL-SPH trained on SC ($n = 3,000$) under size changes.

Metric	SC ($n = 6,000$)		SC ($n = 10,000$)	
	RHF	RL-SPH	RHF	RL-SPH
FR (%) $\uparrow$	100	100	100	100
PG (%) $\downarrow$	21.47 $\pm$ 3.04	0.00 $\pm$ 0.00	20.74 $\pm$ 3.57	0.00 $\pm$ 0.00
PI $\downarrow$	415.3 $\pm$ 26.4	237.1 $\pm$ 14.8	548.2 $\pm$ 31.5	372.8 $\pm$ 20.8

H.2.4. DENSITY CHANGES

Table 15 shows that RL-SPH, trained on SC with the density $\rho = 5\%$, consistently generated feasible solutions across test instances with varying densities. Specifically, RL-SPH achieved a 100% FR on both the sparser dataset ($\rho = 1\%$) and the denser one ($\rho = 10\%$). The results demonstrate that RL-SPH quickly obtained higher-quality solutions than RHF. This indicates that RL-SPH is robust to variations in instance density.

Table 15. Generalization performance of RL-SPH trained on SC ($\rho = 5\%$) under density changes.

Metric	SC ($\rho = 1\%$)		SC ($\rho = 10\%$)	
	RHF	RL-SPH	RHF	RL-SPH
FR (%) $\uparrow$	100	100	100	100
PG (%) $\downarrow$	23.77 $\pm$ 2.33	0.00 $\pm$ 0.00	20.73 $\pm$ 4.47	0.00 $\pm$ 0.00
PI $\downarrow$	80.8 $\pm$ 11.2	43.6 $\pm$ 7.4	233.4 $\pm$ 37.7	113.2 $\pm$ 8.7

H.3. Synergies with Solver-Internal Mechanisms

We examine two ways in which RL-SPH synergizes with standard optimization routines: integration with ILP solvers for accelerated solution discovery (Appendix H.3.1), and interaction with the presolving stage (Appendix H.3.2).

H.3.1. INTEGRATION WITH ILP SOLVERS

We evaluate RL-SPH integrated with SCIP (RL-SPH+SCIP) against two baselines: PAS (Han et al., 2023) integrated with SCIP (PAS+SCIP) and SCIP. Both SCIP and PAS+SCIP are run with their default settings. PAS is trained for the same duration as RL-SPH, taking around 31 minutes on IS, as shown in Appendix G.2. Table 16 presents the optimization performance of the compared methods under a 50-second time limit, where BKS was obtained by SCIP within 1,000 seconds. RL-SPH+SCIP fixes each variable to a unanimous value based on the feasible solutions generated by RL-SPH(LP) during the first five seconds. RL-SPH+SCIP outperformed both SCIP and PAS+SCIP in terms of PG and PI on both benchmarks, achieving high-quality solutions with a PG below 1% relative to the BKS . Unlike PAS, RL-SPH is capable of generating feasible solutions independently, allowing it to safely fix more variables using an ensemble of feasible solutions, which results in a smaller problem size. Moreover, RL-SPH supports non-binary integer variables with a $2.3\times$ lower PI than SCIP, whereas PAS does not support such variables (NBI in Table 16). Although reducing the problem size can significantly

accelerate the discovery of high-quality solutions (Nair et al., 2020), it may also introduce sub-optimality. Given that the primary goal of primal heuristics is to quickly find high-quality feasible solutions (Berthold, 2006; Cantürk et al., 2024), exact optimality is often considered a secondary concern. Nevertheless, improving the optimality within RL-SPH could be a promising direction for future research (see Appendix I.8), as it remains an open challenge of ML for CO (Son et al., 2023).

Table 16. Solving performance of RL-SPH integrated with SCIP.

Dataset	Metric	SCIP	PAS+SCIP	RL-SPH+SCIP
IS	FR (%) $\uparrow$	100	100	100
	PG (%) $\downarrow$	1.24 $\pm$ 1.01	0.30 $\pm$ 0.61	0.14$\pm$0.13
	PI $\downarrow$	2.4 $\pm$ 0.3	1.7 $\pm$ 0.4	0.5$\pm$0.5
NBI	FR (%) $\uparrow$	100		100
	PG (%) $\downarrow$	0.24 $\pm$ 0.03	<i>not applicable</i>	0.23$\pm$0.04
	PI $\downarrow$	11.5 $\pm$ 2.3		4.9$\pm$0.9

We conducted an additional experiment by combining RL-SPH with the commercial solver Gurobi. This experiment followed the same protocol as in Table 16. To ensure a fair comparison with SCIP, Gurobi was configured to use a single thread as done in the previous work (Han et al., 2023), with all other parameters kept at their default settings. Table 17 presents the results on NBI, where RL-SPH+Gurobi denotes RL-SPH integrated with Gurobi. The experimental results show that RL-SPH converged to *BKS* faster when integrated with Gurobi, aligning with the findings presented in Table 16.

Table 17. Solving performance of RL-SPH integrated with ILP solvers.

Dataset	Metric	SCIP	Gurobi	RL-SPH+SCIP	RL-SPH+Gurobi
NBI	FR (%) $\uparrow$	100	100	100	100
	PG (%) $\downarrow$	0.25 $\pm$ 0.03	0.03 $\pm$ 0.02	0.24 $\pm$ 0.04	0.03$\pm$0.03
	PI $\downarrow$	11.5 $\pm$ 2.3	5.2 $\pm$ 1.0	4.9 $\pm$ 0.9	2.4$\pm$0.5

H.3.2. EFFECT OF PRESOLVING ON RL-SPH

Enabling presolving in RL-SPH simplifies the given instance by eliminating redundant constraints and variables. For instance, nearly 600 constraints are removed in the IS instances after presolving. Table 18 shows PG of RL-SPH with and without presolving on IS, using a time limit of 30 seconds. The results demonstrate that activating the presolver leads to better solution quality within the same time budget.

Table 18. Effect of presolving on RL-SPH performance on IS. The time limit is set to 30 seconds.

Presolve	RL-SPH(Random)	RL-SPH(LP)
$\times$	4.00 $\pm$ 2.04	1.05 $\pm$ 1.46
$\checkmark$	3.84$\pm$2.30	0.83$\pm$1.13

H.4. Hyperparameter Analysis

H.4.1. EFFECT OF THE TOWARD-OPTIMAL BIAS

Table 19 shows the performance of RL-SPH under different values of the toward-optimal bias α . With a toward-optimal bias (i.e., $\alpha > 1$), RL-SPH achieves better PG, PI, and #win than the value obtained with $\alpha = 1$. In contrast, RL-SPH without such bias (i.e., $\alpha = 1$) consistently attains the lowest #win across all datasets. These results demonstrate that injecting a toward-optimal bias helps the agent reach higher-quality solutions, highlighting the importance of guiding the agent’s search during training. Increasing the bias from $\alpha = 2$ to $\alpha = 5$ does not necessarily yield further improvements. Notably, in SC, the model with $\alpha = 5$ achieves a PG of 2.52%, which is even higher than the value obtained with $\alpha = 1$. Given that the model also exhibits a noticeably high #viol₂ of 11.2 in SC, we conjecture that it becomes overly focused on pursuing a lower objective value (i.e., $obj_{t+1} < obj_t$), which in turn leads to more aggressive attempts that may even violate constraints. Therefore, it is advisable to choose α to be greater than 1 but not excessively large.

Table 19. Performance of RL-SPH under different toward-optimal bias values. #viol_2 is the sum of variable bound violations and linear constraint violations in *phase2*. #win measures the number of test instances in which a method reaches the best value among all compared methods. The time limit is set to 150 seconds for SC and 50 seconds for all other datasets.

Dataset	Toward-optimal bias	FR (%) $\uparrow$	PG (%) $\downarrow$	PI $\downarrow$	#win $\uparrow$	#viol_2
IS	$\alpha = 1$	100	3.41 $\pm$ 2.10	7.2 $\pm$ 1.0	9	1.0 $\pm$ 0.1
	$\alpha = 2$	100	0.94 $\pm$ 1.29	3.4 $\pm$ 0.6	45	0.3 $\pm$ 0.1
	$\alpha = 5$	100	0.85 $\pm$ 1.20	3.3 $\pm$ 0.6	53	0.4 $\pm$ 0.1
CA	$\alpha = 1$	100	3.22 $\pm$ 3.00	12.5 $\pm$ 1.3	17	2.2 $\pm$ 0.5
	$\alpha = 2$	100	0.88 $\pm$ 1.64	11.8 $\pm$ 1.1	57	2.1 $\pm$ 0.4
	$\alpha = 5$	100	2.49 $\pm$ 2.58	12.4 $\pm$ 1.2	26	3.1 $\pm$ 0.5
SC	$\alpha = 1$	100	2.39 $\pm$ 2.13	98.2 $\pm$ 6.8	32	4.0 $\pm$ 4.7
	$\alpha = 2$	100	1.74 $\pm$ 2.22	98.4 $\pm$ 7.2	50	4.7 $\pm$ 6.3
	$\alpha = 5$	100	2.52 $\pm$ 2.41	97.8 $\pm$ 7.2	33	11.2 $\pm$ 17.9
MVC	$\alpha = 1$	100	14.18 $\pm$ 0.90	8.8 $\pm$ 0.4	0	1.7 $\pm$ 0.2
	$\alpha = 2$	100	0.43 $\pm$ 0.69	4.6 $\pm$ 0.3	60	0.8 $\pm$ 0.1
	$\alpha = 5$	100	0.67 $\pm$ 0.83	4.7 $\pm$ 0.4	41	1.1 $\pm$ 0.1
NBI	$\alpha = 1$	100	0.24 $\pm$ 0.11	10.9 $\pm$ 0.6	1	1.4 $\pm$ 0.3
	$\alpha = 2$	100	0.06 $\pm$ 0.07	10.7 $\pm$ 0.6	29	2.7 $\pm$ 0.6
	$\alpha = 5$	100	0.01 $\pm$ 0.03	10.8 $\pm$ 0.7	71	4.6 $\pm$ 0.7

H.4.2. EFFECT OF THE NEIGHBOR VARIABLE RATIO

Table 20 presents the performance of RL-SPH under different ratios of neighbor variables among the changeable variables (i.e., $q/\tilde{n}$). Regardless of the ratio values, RL-SPH finds a feasible solution in all experimental settings. In IS and MVC, higher ratios enable RL-SPH to reach higher-quality feasible solutions more quickly. These results suggest that IS and MVC exhibit stronger interactions among neighbor variables compared with the other datasets. As shown in Table 21, IS and MVC have relatively high variable degrees compared to their constraint degrees (i.e., a higher degree ratio). Consequently, when the agent modifies the value of a seed variable to improve feasibility, it must also account for the many constraints involving that variable, along with the other variables that appear in those constraints. Thus, for datasets with a large degree ratio, a larger neighbor variable ratio is preferable, as it allows the agent to capture interactions among more neighbor variables.

Table 20. Performance of RL-SPH under varying neighbor variable ratios. The time limits are consistent with those in Table 19.

Dataset	Neighbor variable ratio	FR (%) $\uparrow$	PG (%) $\downarrow$	PI $\downarrow$	#win $\uparrow$
IS	0.3	100	8.22 $\pm$ 1.70	6.9 $\pm$ 0.8	0
	0.5	100	5.86 $\pm$ 1.94	5.7 $\pm$ 0.9	0
	0.7	100	0.00 $\pm$ 0.00	2.8 $\pm$ 0.2	100
CA	0.3	100	0.99 $\pm$ 2.08	11.9 $\pm$ 1.2	60
	0.5	100	1.57 $\pm$ 2.11	12.1 $\pm$ 1.2	40
	0.7	100	8.84 $\pm$ 3.38	15.0 $\pm$ 1.6	0
SC	0.3	100	2.15 $\pm$ 2.24	97.6 $\pm$ 6.7	39
	0.5	100	1.59 $\pm$ 2.10	98.2 $\pm$ 7.4	52
	0.7	100	2.55 $\pm$ 2.37	97.2 $\pm$ 7.4	31
MVC	0.3	100	7.49 $\pm$ 0.94	7.8 $\pm$ 0.4	0
	0.5	100	6.90 $\pm$ 0.97	7.6 $\pm$ 0.4	0
	0.7	100	0.00 $\pm$ 0.00	4.8 $\pm$ 0.2	100
NBI	0.3	100	0.03 $\pm$ 0.04	10.9 $\pm$ 0.7	59
	0.5	100	0.05 $\pm$ 0.06	10.7 $\pm$ 0.6	32
	0.7	100	0.09 $\pm$ 0.06	10.9 $\pm$ 0.8	10

Table 21. Average degree of constraints and variables.

Dataset	IS	CA	SC	MVC	NBI
Degree of constraints d_c	2.00	8.31	150.00	2.00	198.98
Degree of variables d_v	7.94	5.56	100.00	7.96	198.98
Degree ratio d_v/d_c	3.96	0.67	0.67	3.98	1.00

H.5. Qualitative Analysis

H.5.1. BEHAVIOR OF THE TWO-PHASE REWARD SYSTEM

To verify that the proposed two-phase reward system operates as intended, we perform an additional analysis. Figure 10 illustrates the optimization trajectory of RL-SPH over 100 search steps on instances randomly sampled from each dataset. As described in Section 3.1.3, the primary goal of *phase1* is to repair infeasible solutions and obtain the first feasible one. Across all datasets, the initial solutions violate a large number of constraints. For example, the initial solution for MVC violates approximately 3,000 constraints. RL-SPH gradually reduces the number of violated constraints and eventually satisfies all of them, thereby discovering the first feasible solution.

According to the policy defined in Section 3.1.3, *phase2* begins once the first feasible solution is found. Since the goal of *phase2* is to improve the incumbent objective value obj_b , the agent searches for higher-quality feasible solutions within its neighborhood (see Algorithm 1). As shown in Figure 10, after transitioning into *phase2*, the agent takes moves that preserve feasibility while improving obj_b . Across all datasets, the RL-SPH agent exhibits a consistent behavioral pattern, providing evidence that it operates in accordance with the intended design of our reward system.

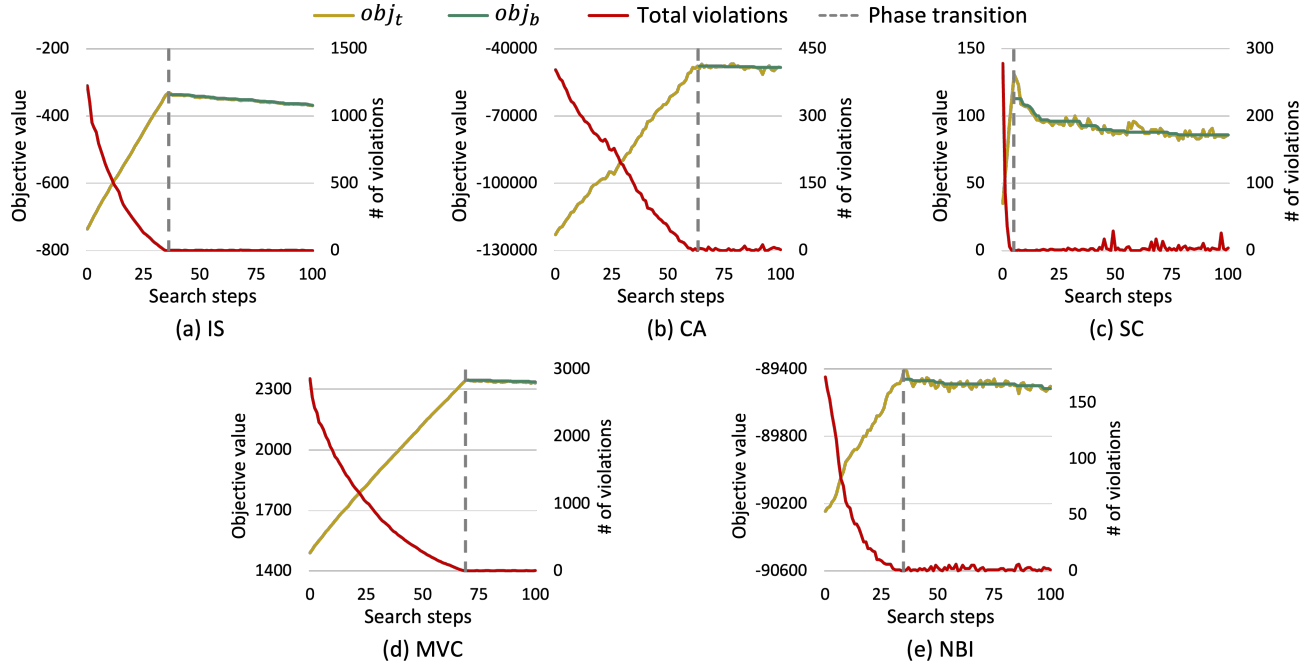


Figure 10. Illustration of RL-SPH’s optimization trajectory over search steps, using one randomly sampled instance from each dataset. The total number of violations is computed as the sum of variable bound violations and linear constraint violations. The period before the phase transition corresponds to *phase1*, and the period after the transition corresponds to *phase2*.

H.5.2. CAPTURING LONG-RANGE VARIABLE DEPENDENCIES

To examine whether our ILP-GT learns long-range dependencies among variables in ILP problems represented as bipartite graphs, we conduct a case study showing attention weights between distant variables. Figure 11 shows a snapshot captured during the RL-SPH’s optimization of a randomly sampled instance from the IS dataset, where interactions among neighbor variables are important as discussed in Appendix H.4.2. Figure 11(a) reports the shortest-path hop distances between variable nodes that are fed into ILP-GT, while Figure 11(b) depicts the attention weights computed among these variable nodes. Although the reward context is also included as input nodes in practice, it is omitted here for consistency with Figure 11(a). As shown in Figure 11(a), variable-node 10 is eight hops away from variable-node 17, yet it receives a larger attention weight than several nearer nodes. This behavior aligns with observations reported in Graph Transformer literature (Wu et al., 2021b) and suggests that our ILP-GT can effectively capture long-range dependencies among decision variables in ILP.

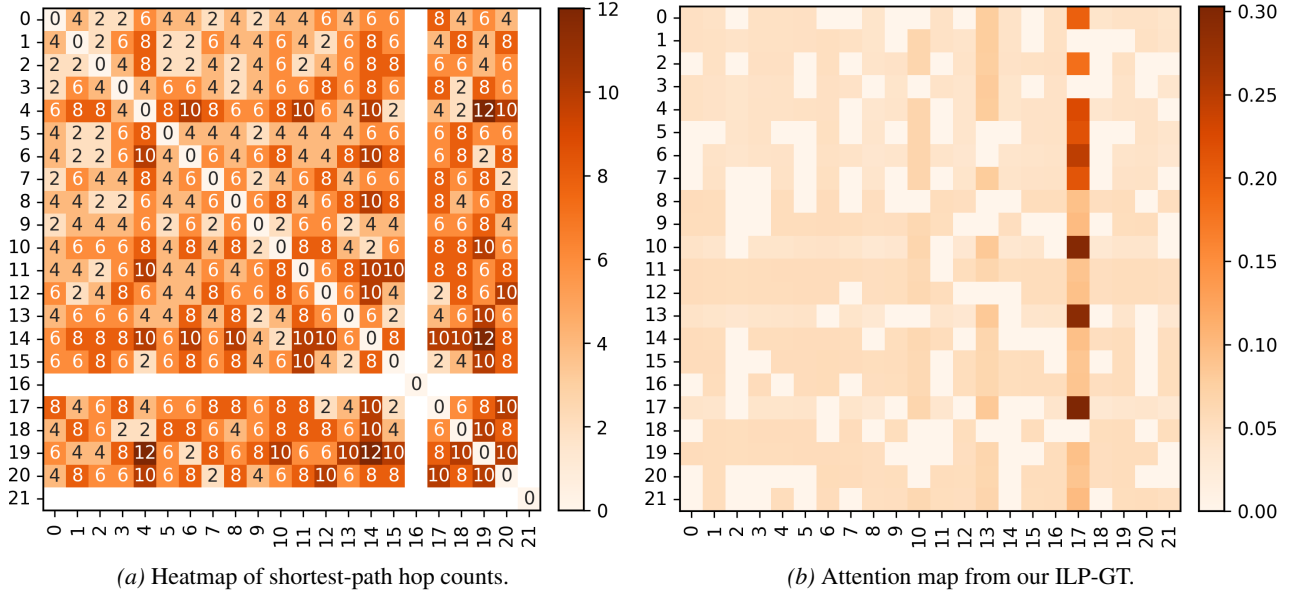


Figure 11. Heatmap of shortest-path hop counts and attention map from our ILP-GT. This example is randomly sampled from the IS test set. The attention map is taken from the final Transformer layer. The x-axis and y-axis correspond to variable nodes. In (a), blank cells in the heatmap indicate that there is no path between the two nodes.

H.5.3. TRAINING STABILITY AND CONVERGENCE

Figure 12 plots the rewards that RL-SPH receives over time during training on each dataset. Since the reward mechanism of RL-SPH consists of *phase1* and *phase2*, the reward magnitude periodically fluctuates whenever the agent transitions to *phase2* or begins *phase1* of the next instances. As described in Section 3.4 and Appendix G.2, the agent stays in *phase1* for one quarter of the total step limit allocated to each instance, which indicates that the period with relatively higher rewards in Figure 12 corresponds to *phase1*. Except for MVC, the training process converges within about 10 minutes and consistently exhibits a repeating and stable pattern.

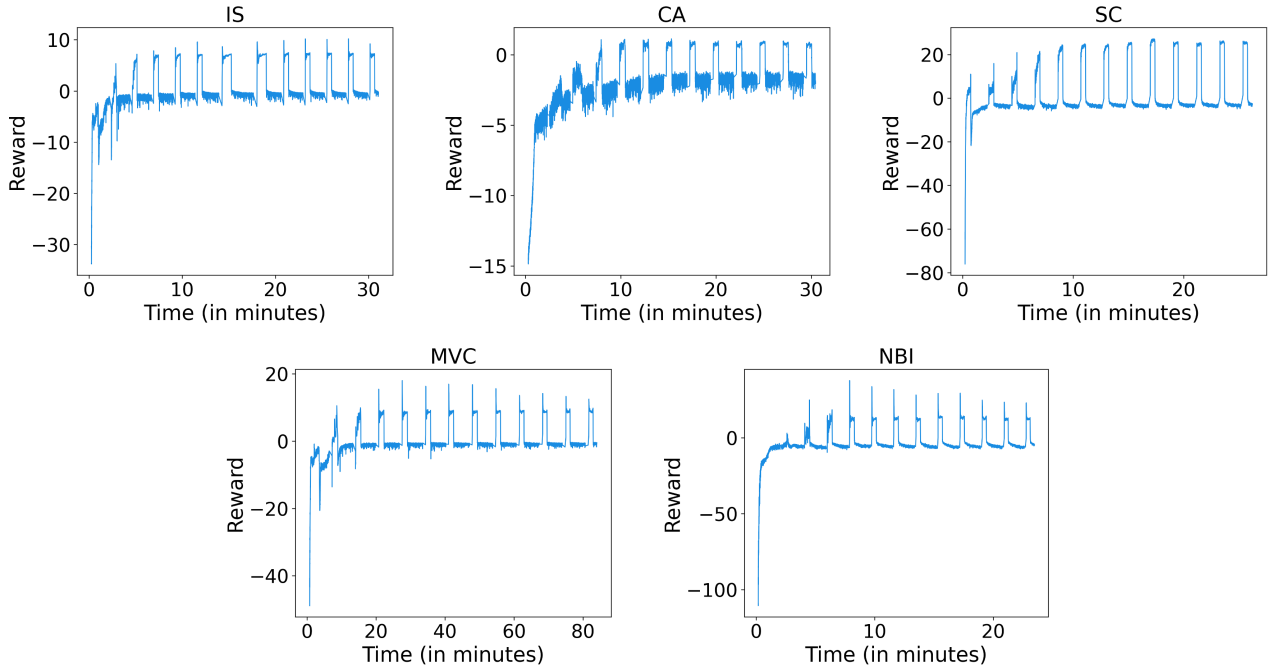


Figure 12. The training curves of RL-SPH on each dataset.

I. Additional Discussion

I.1. Feasibility as a Core Focus of This Work

In this work, our primary focus is on finding feasible solutions from an infeasible start point, rather than optimizing objective values. By definition in Section 2.1, an optimal solution is *feasible* and achieves the lowest objective value. Even if a solution yields the lowest objective value, it is considered invalid if it violates even a single constraint. Therefore, achieving feasibility is a crucial prerequisite for optimality. Furthermore, reaching optimality is recognized as a major challenge in the ML for CO community (Son et al., 2023), which underscores the importance of feasibility in ILP.

Despite this importance, feasibility is not typically guaranteed by primal heuristics (Shoja & Axehill, 2023), including E2EPH methods (Nair et al., 2020; Shen et al., 2021; Yoon, 2022; Han et al., 2023; Cantürk et al., 2024; Huang et al., 2024; Liu et al., 2025a; Zeng et al., 2024; Geng et al., 2025). As discussed in Appendix E, to the best of our knowledge, no existing E2EPH has achieved a 100% feasibility rate. These observations suggest that the *feasibility rate* (FR) should be regarded as a fundamental evaluation metric in this research area. Therefore, we argue that feasibility should be prioritized and that methods achieving higher FR should be regarded as stronger baselines. From this perspective, we consider RHF to be a stronger baseline than other primal heuristics, including the E2EPH methods.

I.2. Training Costs

Most studies on E2EPH are based on supervised learning models (Nair et al., 2020; Yoon, 2022; Han et al., 2023; Huang et al., 2024; Cantürk et al., 2024; Zeng et al., 2024). Obtaining training labels for ILP (e.g., near-optimal solutions) requires a considerable amount of time. For example, such labels are obtained by running Gurobi with a time limit of 3,600 seconds for a single instance (Han et al., 2023; Huang et al., 2024). Considering that training requires on the order of hundreds of instances, this constitutes a substantial computational cost. This labeling process becomes even more computationally expensive for larger instances, limiting the practicality for NP-hard problems (Cantürk et al., 2024). Moreover, training for the supervised learning models usually converges within 5 hours (Huang et al., 2024).

In contrast, RL-SPH does not require near-optimal feasible solutions for training. In addition, all datasets required less than an hour to train RL-SPH, except MVC with 83 minutes, as shown in Table 9. When compared to a representative supervised learning method (Han et al., 2023) in terms of training time, RL-SPH achieved an average speedup of $8.7\times$ on the same datasets, as shown in Table 10. Consequently, RL-SPH incurs lower training costs than supervised learning methods, as it requires no labeling process and less training time. For an unsupervised method (Geng et al., 2025), training on the datasets took about 16 hours, whereas RL-SPH was nearly $34\times$ faster, as shown in Table 10. This substantial reduction in training time underscores the practicality of RL-SPH.

I.3. Training Efficiency

Our feasibility-aware search strategy enables efficient training. This strategy allows the agent to focus on variables that are more likely to improve feasibility by filtering out uninformative inputs. In a real-world analogy, this is akin to preparing for a final exam by focusing on the topics the professor highlighted during lectures, rather than studying an entire 1000-page textbook including bibliography and index. The former one enables targeted learning, which could result in improved exam outcomes. In this sense, this learning strategy can be viewed as a higher-level attention mechanism—operating at the page-level—compared to one that focuses on important words within a page. Likewise, our search strategy narrows the broad observation space to more informative regions, enhancing learning efficiency through denser reward signals. We emphasize this feasibility-aware search strategy as a core contribution, as it provides a systematic way to reduce the search space and enhance the RL agent’s learning efficiency. The insights from this learning strategy may also be useful in other domains, such as RL for routing optimization, which requires several days of training (Son et al., 2024).

I.4. Complexity Analysis of Feasibility-Aware Search

The computational complexity of Algorithm 1 is determined by three components: variable selection, Transformer processing, and observation computation. For variable selection, multiplying the feasibility state vector $\hat{\mathbf{f}}_t \in \mathbb{R}^{m \times 1}$ with the coefficient matrix $\tilde{\mathbf{A}} \in \mathbb{R}^{m \times n}$ requires $O(mn)$ operations. Processing $\tilde{n}^2 + 3$ input tokens with the Transformer incurs a cost of $O(\tilde{n}^2)$ (Katharopoulos et al., 2020). For the observation update, only the changeable variables need to be used for computation, which results in $O(m\tilde{n})$. Therefore, the overall computational complexity of Algorithm 1 is $O(mn + \tilde{n}^2)$. We believe that our strategy represents only one possible way to reduce the search space, and that further optimization is certainly possible. For instance, variable selection currently takes $O(mn)$, which becomes quadratic when $m \approx n$. Addressing this bottleneck remains a promising direction for future research.

I.5. Potential Bottlenecks at Extreme Scales

While our search strategy alleviates the scalability issue by passing only $\tilde{n} (= 2 \lceil \log_2 n \rceil)$ inputs to our ILP-GT, the quadratic complexity of the Transformer’s attention mechanism could become a bottleneck for extremely large-scale problems. For such cases, it would be beneficial to consider attention architectures specifically designed for lower computational complexity, such as RetNet (Sun et al., 2023) or other variants.

Even if the quadratic attention in the Transformer is resolved, the variable selection may become a new bottleneck because its complexity becomes quadratic when $m \approx n$ (see Appendix I.4). Furthermore, if the number of constraints becomes extremely large, projecting the structural information into the hidden size may incur substantial computational cost. This issue could be addressed by explicitly introducing constraint nodes as part of the ILP-GT input nodes, rather than compressing all structural information into a single projected vector. We conducted preliminary experiments to explore this direction and observed promising performance, although a more thorough evaluation is certainly required.

I.6. Handling Nonlinear Constraints

Since our study is scoped to *Integer Linear Programming* (ILP), we did not explicitly consider designing our reward system to handle nonlinear constraints. Nonetheless, we suspect that the reward system might also work in the presence of nonlinearities because the reward is computed based on the change from the previous timestep, regardless of the type of constraints. However, this has not yet been tested and remains to be explored further.

I.7. Changes in Movement Magnitude

Having a larger neighborhood is effective in enhancing search performance (Shaw, 1998). The size of the neighborhood is associated not only with the number of neighbor variables but also with the magnitude of movement. We denote Δ_x as the maximum magnitude of change for any variable. To allow larger movement, RL-SPH can be easily extended by expanding the action space. For example, to set $\Delta_x = 2$, the action space can be expanded to $\{-2, -1, 0, +1, +2\}$. Table 22 summarizes our preliminary experiments on the effect of Δ_x . The model with $\Delta_x = 5$ achieved the highest #win, which leverages the largest action set in this experiment. These results suggest that expanding the movement range enhances the agent’s capability to reach better objective values.

We set $\Delta_x = 1$ in our main experiments to prioritize feasibility over optimality (see Appendix I.1). Notably, RL-SPH even with $\Delta_x = 1$ outperforms four SPH baselines and the SOTA E2EPH (Geng et al., 2025). Thus, increasing action magnitudes would not affect our core conclusions. Nevertheless, this observation points toward a promising avenue for improving solution optimality within RL-SPH, which we discuss further in Appendix I.8.

Table 22. Performance by Δ_x on the NBI dataset with 100 variables and 50 constraints. A 300-second time limit was applied to all cases.

Max Magnitude	FR (%) $\uparrow$	PG (%) $\downarrow$	#win $\uparrow$
$\Delta_x = 1$	100	0.36	21
$\Delta_x = 3$	100	0.31	40
$\Delta_x = 5$	100	0.31	50

I.8. Promising Directions to Improve Optimality

To further improve the optimality within RL-SPH, several research directions can be explored. First, a promising avenue is the dynamic determination of the action space. As observed in Appendix I.7, a larger movement magnitude could improve the objective value. Since the most suitable action space may depend on the agent’s current state, dynamically adapting the action space could be more effective than using a static one. For instance, when the agent is trapped in a deeper local optimum, a larger movement magnitude may be required to escape.

Second, integrating metaheuristics (Hussain et al., 2019), high-level strategies that guide the search process, could further enhance RL-SPH. For example, Iterated Local Search (ILS) (Lourenço et al., 2003) introduces perturbations to escape local optima during the search. In this context, RL-SPH could be extended to detect whether it is stuck in a local optimum and learn how to apply perturbations—in both direction and magnitude—to improve search performance.

Third, RL-SPH could be extended to learn how to select promising neighborhoods. Prior studies on LNS have demonstrated the effectiveness of learning-based neighborhood selection policies (Huang et al., 2023). Building on this line of work, future research could explore how RL-SPH can learn to identify and exploit high-quality neighborhoods.